



COMSATS University Islamabad

Sahiwal Campus



A project presented to
COMSATS University Islamabad, Sahiwal Campus

In partial fulfillment of
requirement for the degree
of

Bachelor of Science in Computer Science (2022-2026)

By

Mohsin Haider

CIIT/SP22-BCS-136/SWL

Muazzam Mushtaq

CIIT/SP22-BCS-189/SWL

Muhammad Mohsan Saeed

CIIT/SP22-BCS-141/SWL

Declaration

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied the report entirely based on our efforts. If any part of this project is proved to be copied from any source or found to be the reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted because of any application for any other degree or qualification of this or any other university or institute of learning.

Mohsin Haider

Muhammad Mohsan Saeed

Muazzam Mushtaq

Certificate of Approval

This is to certify that the final year project of BS(CS) "**AI LogGuard: Intelligent Anomaly Detection and Automated Threat Response in System Logs**" was developed by **Mohsin Haider (CIIT/SP22-BCS-136/SWL)**, **Muhammad Mohsan Saeed (CIIT/SP22-BCS-141/SWL)** and **Muazzam Mushtaq (CIIT/SP22-BCS-189/SWL)** under the supervision of "**Mr. Muhammad Umer**" and co-supervisor "**Dr. Muhammad Shoaib**" and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor
Mr. Muhammad Umer
Lecturer
Department of Computer Science
COMSATS University Islamabad Sahiwal Campus

External Examiner
Dr. Javed Rashid
Assistant professor
Department of Computer Science
University of Okara

Head of Department
Dr. Zafar Iqbal Roy
Assistant Professor
Department of Computer Science
COMSATS University Islamabad Sahiwal Campus

Executive Summary

In the modern digital landscape, the exponential volume and velocity of machine-generated logs have rendered manual analysis and static rule-based detection mechanisms inefficient. Traditional Security Information and Event Management (SIEM) systems often struggle to identify novel, adaptive threats such as zero-day exploits and "low-and-slow" attacks, leading to significant security gaps and analyst fatigue. This project presents "**AI LogGuard**," an intelligent, automated cybersecurity solution designed to bridge the gap between raw log data and actionable security intelligence.

AI LogGuard leverages a hybrid approach to threat detection, combining the **Isolation Forest** unsupervised machine learning algorithm with **TF-IDF** (Term Frequency-Inverse Document Frequency) vectorization to establish dynamic behavioral baselines. This allows the system to detect statistical anomalies and unknown threat patterns in real-time without relying on pre-labeled datasets. The system architecture is built upon a multi-threaded Python framework using **CustomTkinter**, ensuring low-latency processing and a responsive user interface.

To enhance operational efficiency, the system integrates a customizable **Response Rules Engine** capable of executing automated mitigation actions, such as IP blocking and alert generation, thereby reducing the mean time to respond (MTTR). Furthermore, a novel **Large Language Model (LLM) Forensic Analysis** module is implemented to address the "black box" problem of AI; it interprets complex anomaly scores and provides administrators with natural-language summaries and remediation steps. The resulting application offers a scalable, cost-effective, and user-friendly alternative to complex enterprise solutions, significantly enhancing the security posture of Small and Medium Enterprises (SMEs).

Keywords: *Anomaly Detection, Isolation Forest, Cybersecurity, Log Analysis, Large Language Models (LLM), Real-Time Monitoring, Automated Response.*

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of His boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor "**Mr. Muhammad Umer**" and co-supervisor "**Dr. Muhammad Shoaib**". Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

We are also thankful to our parents and family who have been a constant source of encouragement for us and have brought us the values of honesty & hard work.

Mohsin Haider

Muhammad Mohsan Saeed

Muazzam Mushtaq

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
SIEM	Security Information and Event Management
SRS	Software Requirement Specification
SDD	Software Design Description
FR	Functional Requirement
NFR	Non-Functional Requirement
TF-IDF	Term Frequency-Inverse Document Frequency
GUI	Graphical User Interface
LLM	Large Language Model
SQLite	Structured Query Language Lite

Table of Contents

Chapter No 1: Introduction	
1. Introduction	1
1.1 Project Background	1
1.2 Brief	3
1.3 Relevance to Course Modules	4
1.4 Literature Review	5
1.4.1 Limitations of Traditional SIEM Systems	5
1.4.2 Machine Learning in Cybersecurity	5
1.5 Scalable Data Pipelines and Real-Time Processing	6
1.6 Gap Analysis: Existing Solutions vs. AI LogGuard	7
1.7 Analysis from Literature Review	7
1.8 Methodology and Software Lifecycle for This Project	8
1.8.1 Rationale behind Selected Methodology	8
1.8.2 The Iterative Development Cycles	9
Chapter No 2: Problem Definition.....	
2. Problem Definition	11
2.1 Problem Statement	12
2.1.1 The Log Volume and Velocity Crisis	13
2.1.2 Failure of Static Detection Against Adaptive Threats.....	14
2.1.3 Operational Drain and Analyst Fatigue	15
2.2 Deliverable and Development Requirements	16
2.2.1 Development Requirements (NFRs):	17
2.3 Current System.....	18
2.3.1 Manual Log Analysis: Inherently Unscalable and Prone to Error	18
2.3.2 Basic SIEM / Rule-Based Tools: Technologically Rigid and Operationally Noisy	19

Chapter No 3: Requiements Analysis	
3. Requirement Analysis	20
3.1 Use Cases Diagrams	20
3.1.1 Actors.....	21
3.1.2 Main Use Case:.....	21
3.2 Detailed Use Case	22
3.2.1 Use Case 1: Monitor Logs in Real-Time	22
3.2.2 Use Case 2: AI Forensic Analysis (LLM Integration)	23
3.2.3 Use Case 3: Configure Response Rules	24
3.3 Functional Requirements	24
3.4 Non-Functional Requirements	27
Chapter No 4: Design and Architecture	
4. Design and Architecture.....	29
4.1 System Architecture	29
4.1.1 Architectural Components	30
4.2 Data Representation	33
4.3 Process Flow/Representation	34
4.4 Design Models.....	38
4.4.1 Sequence Diagram	38
4.4.2 Scenario 1: Real-Time Anomaly Detection & Alerting	39
4.4.3 Scenario 2: LLM Forensic Analysis.....	40
Chapter No 5: Implememtation	
5. Implementation	42
5.1 Algorithm	42
5.1.1 Log Parsing and Preprocessing.....	42
5.1.2 Feature Engineering (TF-IDF Vectorization)	42
5.1.3 Anomaly Detection (Isolation Forest)	42

5.1.4	Automated Response Rule Engine	43
5.2	External APIs	45
5.3	User Interface.....	46
5.3.1	Component Implementation	46
5.3.2	Key Page Interfaces	47
Chapter No 6: Testing and Evaluation		
6.	Testing and Evaluation	55
6.1	Manual Testing.....	55
6.1.1	System Testing.....	55
6.1.2	Unit Testing	56
6.1.3	Functional Testing	57
6.1.4	Integration Testing	58
6.2	Automated Testing	59
Chapter No 7: Conclusion and Future Work		
7.	Conclusion and Future Work.....	61
7.1	Conclusion.....	61
7.1.1	Achievement of Core Objectives.....	61
7.1.2	Summary of Technical Success and Innovation	62
7.2	Future Work	63
7.2.1	Evolution of the Anomaly Detection Core	63
7.2.2	Scaling the Ingestion Pipeline and Persistence Layer	64
7.2.3	Achieving True Autonomous Remediation	64
7.2.4	User Experience and Operational Intelligence	65
References.....		67

List of Tables

Table 1.1 Relevance to course Modules.....	4
Table 1.2: Gap Analysis.....	7
Table 2.1 Development Requirements.....	17
Table 3.1 Monitor Logs in Real-Time.....	22
Table 3.2 AI Forensic Analysis.....	23
Table 3.3 Configure Response Rules	24
Table 5.1 External APIs	46
Table 6.1 System Testing	55
Table 6.2 Unit Testing 1	56
Table 6.3 Unit Testing 2	57
Table 6.4 Function Test 1	58
Table 6.5 Integration Test.....	59
Table 6.6 Automated Testing.....	60

List of Figures

Figure 1.1: Agile/Iterative Model	9
Figure 3.1 Ai LogGuard Use Case Diagram	20
Figure 4.1 AI LogGuard Architecture	29
Figure 4.2 AI LogGuard Data Representation	34
Figure 4.3 Anomaly Detection	39
Figure 4.4 LLM Forensics	40
Figure 4.5 Class Diagram	41
Figure 5.1: Dashboard Interface.....	47
Figure 5.2: Live Monitor Interface.....	48
Figure 5.3: Response Rules Editor Interface	49
Figure 5.4: AI Forensics interface	50
Figure 5.5: Alerts & Anomalies Interface	51
Figure 5.6: Threat Intel Interface	52
Figure 5.7: Reports Page Interface	53
Figure 5.8: Settings page Interface.....	54

Chapter 1

Introduction

1. Introduction

The modern digital landscape is characterized by an unprecedented volume of machine-generated data. As software architectures evolve into complex, distributed systems, the reliance on system logs for observability, security, and debugging has become critical. However, the sheer velocity and volume of these logs often render manual analysis inefficient and error prone.

Against this backdrop, this project presents "**AI LogGuard**," an advanced log analysis system designed to bridge the gap between raw data and actionable intelligence. This document establishes the operational context of the project, detailing how it leverages cutting-edge Artificial Intelligence to automate the interpretation of system logs. Furthermore, it explicitly outlines the project's alignment with the Bachelor of Science in Computer Science (BSCS) curriculum. By synthesizing core competencies including Software Engineering, Database Management, and Artificial Intelligence AI LogGuard demonstrates the practical application of theoretical computer science concepts.

1.1 Project Background

Modern organizations generate an unprecedented volume of system, network, and application logs as a natural byproduct of daily operations. These logs ranging from firewall and intrusion detection alerts to authentication attempts and application-level events form the foundational evidence required to detect cyberattacks, perform forensic investigations, and maintain regulatory compliance. However, the rapid expansion of distributed architectures, cloud platforms, and interconnected services has caused log data to increase in both velocity and structural complexity, overwhelming traditional monitoring techniques.

Historically, system administrators and security analysts have relied on manual command-line tools and static rule-based systems to interpret log data. While effective for predictable or well-known attack patterns, these approaches fail in modern threat environments characterized by polymorphic malware, Living-off-the-Land (LotL) techniques, and low-and-slow intrusions that operate below fixed detection thresholds. Manual analysis is not only inherently slow and error-

prone but also fundamentally unscalable, leading to delayed detection and incomplete forensic visibility.

At the same time, Security Operations Centers (SOCs) face a persistent operational challenge: alert fatigue. Traditional SIEM tools often generate overwhelming numbers of low-context, false-positive alerts, requiring analysts to expend disproportionate effort validating benign events. This not only reduces productivity but also increases the risk of real threats being overlooked.

AI LogGuard was conceived as a response to these systemic challenges. The project leverages Machine Learning based anomaly detection, intelligent log parsing, and a real-time monitoring dashboard supported by a modern GUI to deliver a practical, adaptive solution. By integrating a statistical anomaly detection model (Isolation Forest), a dynamic response-rule engine, and an LLM-assisted forensic analysis module, AI LogGuard shifts security monitoring from reactive, signature-based detection to proactive, behavior-driven intelligence.

The system's design emerged from the need to:

- Handle high-volume, high-velocity log streams without latency.
- Provide accurate, adaptive threat detection beyond static rules.
- Reduce analyst workload by minimizing false positives and automating contextual analysis.
- Offer a user-friendly, visual interface suitable for students, researchers, and enterprise trainees.

Ultimately, AI LogGuard serves as both a research-driven academic project and a practical security tool, demonstrating how artificial intelligence and modern UI/UX principles can significantly enhance log monitoring, threat detection, and operational efficiency in cybersecurity environments.

1.2 Brief

AI LogGuard is a dedicated desktop application designed to enhance cybersecurity monitoring through the automated ingestion and analysis of system and application logs. The primary objective of this project is to deprecate inefficient manual workflows in favor of an automated, Machine Learning-driven approach. By moving away from static rule-based detection, the system offers an adaptive solution capable of identifying evolving threat patterns.

The resulting application is a fully functional, multi-threaded software solution developed using **Python 3.13.7** and comprising three primary modules:

1. **Real-time Anomaly Detection Module:** This module utilizes the Isolation Forest algorithm to process incoming log streams with low latency. It assigns dynamic anomaly scores to log entries in real-time, instantly flagging behavior that falls outside of established statistical baselines.
2. **Automated Response & Mitigation Engine:** To ensure rapid defense, a customizable Rule Engine evaluates the severity of detected anomalies. It is capable of executing predefined countermeasures, such as firewall updates or IP blocking, without requiring immediate human intervention.
3. **Generative AI Forensic Intelligence:** By integrating with an external Large Language Model (LLM) service, the application transcends simple alerting. It provides deep semantic analysis of log events, offering administrators human-readable summaries and step-by-step remediation plans for complex security scenarios.

Development Methodology & Stack The project adhered to the Agile/Iterative Model, allowing for continuous refinement of the UI/UX and detection algorithms. The backend logic relies on Python, utilizing Scikit-learn and TensorFlow for predictive modeling, while the frontend is constructed using CustomTkinter to ensure a modern, responsive user experience across platforms. The subsequent chapters of this report provide a granular analysis of the system design (featuring Class and Sequence Diagrams) and the comprehensive testing strategies employed.

1.3 Relevance to Course Modules

The development of the AI Log Guard project required the practical application of several key modules studied during the degree program, demonstrating the integration of theoretical knowledge into a high-impact, real-world security solution.

Table 1.1 Relevance to course Modules

Course Module	Application in AI LogGuard
Artificial Intelligence	Directly applied principles of unsupervised learning and dimensionality reduction to select, train, and deploy the Isolation Forest and TF-IDF models for log anomaly detection.
Data Structures & Algorithms	Utilized efficient data structures like Deques (deque) for thread-safe queues and log buffers, and Counters for real-time statistical aggregation in the dashboard logic.
Database Systems	Designed a schema within SQLite to store structured Traffic Logs, user accounts, Response Rules, and Forensic Analysis reports, ensuring data persistence and integrity.
Operating Systems / Networks	Applied fundamental understanding of log formats (Syslog, Windows Event Logs) and network concepts (IP addresses, CIDR, latency) for log parsing and heuristic scoring.
Software Engineering	Employed Agile and Iterative methodologies, focusing on modularity, comprehensive documentation (SRS, SDD), and continuous integration of decoupled components.
Information Security	Synthesized theoretical knowledge from domains such as IDS, Incident Response, Forensics, and Secure Engineering into a functional, real-world tool.

1.4 Literature Review

The evolution of cybersecurity monitoring has been driven by the increasing complexity of cyber threats and the exponential growth of data. This chapter reviews the existing literature regarding log analysis, the limitations of traditional Security Information and Event Management (SIEM) systems, and the emerging role of Artificial Intelligence (AI) in threat detection. Specifically, it examines the shift from static signature-based methods to unsupervised machine learning techniques, establishing the theoretical basis for selecting the Isolation Forest algorithm for this project.

1.4.1 Limitations of Traditional SIEM Systems

For decades, organizations have relied on rule-based SIEM systems to monitor network activity. These systems function by matching incoming log entries against a database of predefined signatures or correlation rules. While effective against known threats, literature consistently highlights that traditional SIEM rule engines are rigid and slow to adapt.

A primary criticism in recent studies is the reliance on human expertise for rule tuning. As noted by Ganti et al., this dependency creates a reactive security posture; a new rule can only be created *after* a threat has been identified and analyzed. This lag contributes significantly to the "dwell time" of attackers within compromised networks. Furthermore, static rules generate high volumes of false positives when legitimate user behavior fluctuates, leading to the "alert fatigue" phenomenon described in the problem statement.

1.4.2 Machine Learning in Cybersecurity

To address the shortcomings of static detection, the field has seen a paradigm shift toward Artificial Intelligence. Machine Learning (ML) models offer the capability to learn patterns from data rather than relying on explicit programming.

1. Unsupervised Learning for Anomaly Detection

Within the domain of log analysis, researchers strongly favor unsupervised learning techniques such as Clustering, Autoencoders, and Isolation Forests over supervised methods. The rationale for this preference is twofold:

- **Scarcity of Labeled Data:** Cyberattacks are rare events compared to the volume of benign traffic, making it difficult to construct balanced, labeled datasets for supervised training.
- **Novelty Detection:** Supervised models are limited to detecting attacks they have been trained on. Unsupervised models, conversely, define a baseline of "normal" behavior and flag any statistical deviation, enabling the detection of zero-day exploits.

2. The Isolation Forest Algorithm

Among unsupervised algorithms, the **Isolation Forest** has emerged as a superior choice for high volume log streams. Yadav and Singh highlight its distinct advantage: unlike distance-based methods (e.g., K-Means) that require expensive computation to measure the distance between all data points, Isolation Forest detects anomalies by randomly selecting a feature and splitting the data. Anomalies, being few and different, are "isolated" much faster (i.e., with fewer splits) than normal points. This results in a linear time complexity, making it highly robust and efficient for high-dimensional datasets and real-time processing.

1.5 Scalable Data Pipelines and Real-Time Processing

The velocity of modern log generation demands not just intelligent algorithms but also scalable software architectures. Effective solutions must integrate ML models directly into high-throughput data pipelines to minimize latency.

Recent studies emphasize that Python based frameworks like **Scikit-learn** and **TensorFlow**, when combined with non-blocking threading mechanisms, are essential for meeting the strict performance requirements of modern security tools. This literature validates the architectural

decision to use multi-threading and optimized data structures (such as deque) in the development of AI LogGuard.

1.6 Gap Analysis: Existing Solutions vs. AI LogGuard

Based on the reviewed literature, current market solutions generally fall into two categories: expensive enterprise SIEMs (like Splunk or IBM QRadar) that are complex to deploy, or basic open-source tools that lack intelligent features.

Table 1.2: Gap Analysis

Feature	Traditional SIEMs	AI LogGuard (Proposed)
Detection Logic	Static Signatures (Reactive)	Adaptive AI (Isolation Forest)
Response Time	High latency due to batching	Real-time (<100ms)
False Positives	High (Alert Fatigue)	Low (Statistical baselining)
Usability	Complex, steep learning curve	User-friendly GUI + LLM Explanations
Explainability	"Black Box" alerts	Generative AI Forensic Reports

AI LogGuard addresses these identified gaps by democratizing access to advanced AI detection. By combining the speed of the Isolation Forest with the explainability of Large Language Models (LLMs), the system resolves the "Black Box" problem cited in research, making advanced anomaly detection accessible and actionable for security analysts.

1.7 Analysis from Literature Review

Based on the documented literature, **AI Log Guard**'s design offers specific advantages over the shortcomings of conventional systems:

- i. **Adaptive vs. Static Detection:** The system directly implements the recommendation for adaptive AI, utilizing Isolation Forest and heuristic scoring (perform detection in core_logic.py) to bypass the limitations of static rules. This allows for the immediate flagging of novel anomalies.

- ii. **Performance Optimization:** By implementing a multi-threaded Python core (core_logic.py) using non-blocking queues (deque), the system is designed for a low-latency pipeline. This directly targets the performance constraint of processing logs in near real-time, moving beyond batch processing solutions.
- iii. **Usability and Actionability:** While ML models can be complex ("black box" problem), AI Log Guard integrates the LLM Forensic Analysis module (llm_service.py). This feature resolves the "explainability" challenge cited in research by translating complex anomaly scores into natural language summaries and explicit recommended actions, making the AI output immediately useful to the security analyst.
- iv. **Extensibility:** The modular architecture (NFR-C1) and reliance on open-source frameworks (Python, SQLite, Scikit-learn) align with the trend for cost-effective and scalable security solutions, making the tool accessible to Small to Medium-sized Enterprises (SMEs) which typically cannot afford full enterprise SIEM deployments.

1.8 Methodology and Software Lifecycle for This Project

The development of the AI LogGuard project followed a hybrid software development methodology, integrating the core flexibility of **Agile** practices with the structured, incremental delivery of the **Iterative Model**. This approach was strategically selected to manage the inherent technical risks associated with integrating experimental AI models while maintaining a focus on continuous functional delivery.

1.8.1 Rationale behind Selected Methodology

The Agile/Iterative methodology was deliberately chosen to address the unique risks associated with building an AI-driven cybersecurity system.

- **Model Risk Mitigation:** The iterative nature enabled the team to repeatedly evaluate and refine the Isolation Forest model's performance, particularly reducing the False Positive Rate (FPR), ensuring that algorithmic weaknesses were identified and corrected early.

- **Handling Requirement Volatility:** Agile's built-in feedback loops allowed for fast adjustments to critical components such as the Log Parser and Feature Extraction Pipeline, ensuring the system remained adaptable to emerging threats and updated log structures.
- **Emphasis on Usability:** This approach facilitated early development and continuous refinement of the Graphical User Interface (GUI), ensuring the final system is functional and user-friendly.



Figure 1.1: Agile/Iterative Model

1.8.2 The Iterative Development Cycles

The development followed a structured Iterative Development Model, where each cycle produced a functional, validated, and incrementally improved component of the system:

i. **Iteration 1: Core Log Parsing and SQLite Persistence**

- **Focus:** Establishing the fundamental data pipeline.
- **Outcomes:** Development of the log parsing engine, implementation of regular-expression-based extraction for metadata, and deployment of the **SQLite persistence layer** (`database_manager.py`).

ii. **Iteration 2: Isolation Forest Integration and Model Training**

- **Focus:** Introducing the system's AI-driven anomaly detection capability.
- **Outcomes:** Integration of the **Isolation Forest** model within `core_logic.py`, development of vectorization and feature extraction using **TF-IDF**, and initial model training and evaluation.

iii. **Iteration 3: GUI Development and Real-Time Monitoring**

- **Focus:** Usability and real-time operational capabilities.
- **Outcomes:** Construction of the **CustomTkinter-based GUI**, implementation of Live Monitoring for continuous streaming, and integration of the Alert Manager.

iv. **Iteration 4: Advanced Features — Response Rules and LLM Forensic Analysis**

- **Focus:** High-level automation and intelligence components.
- **Outcomes:** Deployment of the **Response Rule Engine**, supporting automated actions like IP blocking, and integration of **LLM-powered forensic analysis**.

Chapter 2

Problem Definition

2. Problem Definition

Modern computer systems continuously generate extensive logs that capture critical details about user activity, system operations, and potential security events. These logs are essential for detecting cyberattacks such as brute-force attempts, port scanning, unauthorized access, malware activity, and abnormal system behavior. However, the sheer volume, velocity, and unstructured nature of log data make manual monitoring nearly impossible. Security analysts often struggle to identify meaningful patterns, correlate events, and respond to threats in a timely manner.

Traditional log monitoring tools are limited by several key issues:

1. **Static rule-based detection** that fails to identify new, evolving, or zero-day attack patterns.
2. **Lack of real-time analysis**, resulting in delayed responses to active threats.
3. **High dependency on manual investigation**, causing slow incident evaluation and increased workload.
4. **Limited integration of machine learning**, reducing the accuracy of anomaly detection.
5. **Complex or expensive commercial tools**, making them unsuitable for academic use, small organizations, or local deployments.

There is a need for an intelligent, automated system that can analyze logs in real time, detect anomalies using AI techniques, enforce security policies automatically, and provide clear insights to users through an accessible interface.

AI LogGuard aims to address these limitations by providing a machine learning driven log analysis and threat detection platform that integrates:

- Real-time monitoring,
- AI-powered anomaly detection,
- Automated rule-based responses,

- Threat intelligence lookup, and
- A simple, intuitive GUI for security analysts.

Thus, the problem is defined as the design and development of a scalable, intelligent log monitoring system capable of automatically detecting, analyzing, and responding to security threats using AI and automation techniques.

2.1 Problem Statement

Modern organizations generate massive volumes of system, application, and network logs every second. These logs contain critical information about system activities, user behaviour, and potential security threats. However, manually reviewing and analyzing such high-volume log data is impractical, time-consuming, and prone to human error. Traditional log monitoring tools often rely on static signatures, lack real-time intelligence, and fail to detect sophisticated or unknown attack patterns. As a result, security teams struggle to identify anomalies such as brute-force attempts, privilege escalation, unauthorized access, and malicious IP activity.

There is a need for an intelligent, automated, and user-friendly solution that can monitor logs in real time, detect abnormal patterns using machine learning, and provide meaningful alerts to security analysts. Furthermore, most existing solutions are either too complex, too expensive, or not customizable for academic, small-scale, or local deployment environments.

AI LogGuard addresses this gap by providing a lightweight, AI-powered log monitoring and anomaly detection system capable of:

- Real-time ingestion and analysis of log streams,
- Automatic detection of suspicious or abnormal behaviour using Machine Learning,
- Intelligent interpretation of logs using an integrated language model,
- Automated response actions through a configurable rule engine, and
- A user-friendly graphical interface for visualization, investigation, and reporting.

This project aims to solve the challenge of efficient log analysis and threat detection by combining AI-based anomaly detection, threat intelligence enrichment, automated response mechanisms, and an intuitive dashboard enabling faster, smarter, and more effective security monitoring.

2.1.1 The Log Volume and Velocity Crisis

Modern IT infrastructures generate logs at unprecedented volume and speed, creating a data environment where manual analysis is no longer feasible. High-density cloud workloads, distributed microservices, containerized applications, and continuous authentication events collectively produce a massive, high-velocity log stream. This rate of generation demands multi-threaded, parallel processing to ensure real-time detection and response. The AI LogGuard system meets this requirement through its asynchronous ingestion pipeline and optimized queue-based architecture.

The implications of this data deluge extend far beyond processing speed. First, organizations are often forced to adopt aggressive data retention and deletion policies due to limited storage capacity. These policies frequently fall short of regulatory and internal compliance standards particularly those requiring long-term log preservation for forensic investigations. The premature deletion of historical logs severely hinders the ability to reconstruct multi-stage attacks and understand the adversary's kill chain.

Second, the overwhelming log volume forces many monitoring tools to rely on data sampling, which statistically increases the likelihood of missing rare yet critical events. These low-frequency anomalies failed authentication bursts, lateral movement attempts, or privilege escalation precursors are often the earliest indicators of advanced threats. Losing them due to sampling creates blind spots in an organization's security posture.

The requirement to process 1 MB of log data within 5 seconds (NFR-P1) is therefore not simply a performance target; it is the minimum operational threshold needed to prevent backlog accumulation. If logs accumulate faster than they can be analyzed, delays propagate through the

entire detection pipeline, resulting in alerts being generated minutes or even hours too late, effectively neutralizing any real-time protection.

Furthermore, the complexity is compounded by the multi-source nature of modern logs. Ingestion pipelines must normalize fundamentally different formats simultaneously for example, converting unstructured Apache access logs alongside structured and sometimes proprietary Windows Event Logs. Without robust normalization and parallel processing, inconsistencies in structure, timestamp formats, and field semantics can lead to misclassification, failed correlation, and incomplete threat detection.

2.1.2 Failure of Static Detection Against Adaptive Threats

Traditional security monitoring systems depend heavily on predefined signatures, correlation rules, and fixed threshold conditions. This creates a systemic and well-documented weakness: static logic cannot detect novel, polymorphic, or stealthy attacks that do not match known patterns. As cyber-attacks continue to evolve, adversaries deliberately craft behaviors that circumvent these rigid rule-based mechanisms. The AI LogGuard system directly addresses this limitation by establishing a dynamic behavioral baseline using machine learning models capable of adapting to emerging patterns.

A fundamental limitation of rule-based detection is its complete blind spot toward “Living off the Land” (LotL) attacks, where adversaries exploit legitimate, trusted system binaries (e.g., *PowerShell*, *wmic.exe*, *certutil*, *rundll32*) to execute malicious actions. Since these binaries are inherently authorized and commonly used, conventional systems classify their activity as benign unless the exact command string appears in a known threat signature. Consequently, malicious operations blend seamlessly with normal activity. AI LogGuard overcomes this constraint by applying anomaly detection models that analyze frequency, sequence patterns, parameter anomalies, and temporal deviations, enabling it to identify subtle behavioral irregularities even when the executable is legitimate.

Furthermore, static correlation rules are fundamentally incapable of detecting low-and-slow attacks, where adversaries intentionally distribute malicious actions over extended time windows to avoid triggering volume-based thresholds. A typical example is spreading Remote Desktop Protocol (RDP) login attempts hours apart, ensuring no alert fires. The statistical nature of AI LogGuard's *Isolation Forest* model operating on multi-dimensional feature density rather than fixed limits—enables the detection of these subtle, distributed anomalies. This temporal and contextual sensitivity ensures early identification of lateral movement attempts long before the attacker escalates privileges or compromises additional systems.

In summary, the failure of static detection mechanisms against adaptive threats creates a critical security gap. AI LogGuard resolves this by integrating machine learning driven anomaly detection that continuously adapts to environmental behavior, thereby providing high-fidelity alerts for both known and unknown attack vectors.

2.1.3 Operational Drain and Analyst Fatigue

Security Operations Center (SOC) teams face an ever-growing operational burden driven by the relentless influx of low-quality, false-positive alerts. This continuous noise leads directly to alert fatigue, a condition in which analysts become desensitized and increasingly prone to overlooking or misclassifying real threats. The problem is further exacerbated by the fragmented nature of modern log ecosystems, where events are generated in diverse formats and originate from distributed, heterogeneous sources. This fragmentation complicates correlation efforts, increases cognitive load, and directly elevates the mean time-to-detection (MTTD).

The operational impact of this fatigue is not merely anecdotal it is empirically measurable. Industry assessments consistently report that analysts spend over 50% of their working hours investigating alerts that ultimately prove benign, representing a severe drain on organizational productivity and cybersecurity posture. The root cause of this inefficiency lies in the poor contextualization of traditional alerts. When a rule-based system generates an alert, it often includes minimal metadata, forcing analysts into time-consuming, repetitive forensic routines.

These routines typically involve manual IP reputation checks, cross-referencing log sources, reconstructing event sequences, and searching for related anomalies across multiple systems.

This manual forensic loop substantially increases the Mean Time to Respond (MTTR), leaving critical infrastructure exposed for longer periods and increasing the probability of successful lateral movement or data exfiltration by an adversary.

AI LogGuard directly addresses this challenge through the integration of an LLM-driven Forensic Analysis Module, which synthesizes contextual information in real time. Instead of presenting analysts with raw, isolated log entries, AI LogGuard provides narrative summaries, root-cause indicators, attack-path hypotheses, and actionable mitigation recommendations. This advanced contextualization dramatically reduces cognitive load, eliminates repetitive manual triage steps, and significantly accelerates both detection and response workflows. By improving analyst efficiency and reducing MTTR, AI LogGuard directly enhances organizational resilience against evolving cyber threats.

2.2 Deliverable and Development Requirements

The project delivers a fully functional, AI-powered desktop application, rigorously validated against specific constraints.

Key Deliverables:

- **AI Anomaly Detection Engine** (Isolation Forest/TF-IDF): Provides adaptive statistical profiling to counter unknown threats. This component is the primary implementation of the AI Module (FR-04), transforming raw log text into actionable Anomaly Scores via Scikit-learn's efficient implementation of TF-IDF and Isolation Forest. The high performance and low computational overhead of Isolation Forest is key to enabling real-time detection on standard analyst hardware.
- **Real-Time Monitoring Pipeline (Multi-threaded)**: Ensures high-throughput, low-latency ingestion. This pipeline, built around Python's deque and threading mechanisms, is

designed to consume logs asynchronously, explicitly maintaining GUI responsiveness while the CoreLogic thread performs intensive ML inference. This architecture prevents UI lockup during peak data loads.

- **Automated Response System:** Manages customizable rules for policy enforcement and active mitigation. This Rule Manager (FR-05, FR-06) uses priority-based logic (set by the user) to determine the appropriate response action—from simulated IP blocking (execute_rule_action in core_logic.py) to external event logging, enforcing security policies dynamically.
- **LLM Forensic Analysis Module:** Augments raw ML scores with expert contextual analysis. This **Forensics Tool** (FR-08) is critical for resolving the explainability issue inherent in complex ML models, offering detailed vulnerability narratives, clear remediation steps, and estimated attack impact.

2.2.1 Development Requirements (NFRs):

The project's success is tied to meeting these specific Non-Functional Requirements (NFRs), which collectively ensure product quality, reliability, and security integrity.

Table 2.1 Development Requirements

Category	ID	Metric / Constraint	Rationale and Implementation Detail
Performance	NFR-P1	Process 1MB log files in 5 seconds for real-time detection.	This constraint validates the architectural choice. Compliance is ensured through multi-threading and the rapid, linear-time inference speed of the Isolation Forest model, specifically avoiding slower deep learning or clustering methods.
Security	NFR-S1	Encrypt all log files during processing and storage. Must meet	Mandates a robust security posture, including secure password hashing for user authentication and thread-safe locking mechanisms in the database to

		ISO 27001 and NIST guidelines.	prevent write corruption (e.g., SQLite's <code>_DB_LOCK</code> in <code>database_manager.py</code>).
Compatibility	NFR-C2	Compatible with Windows and Linux , utilizing Python and CustomTkinter for the GUI.	Guarantees broad accessibility across diverse analyst environments. Python's cross-platform capabilities simplify deployment across varying desktop environments.
Usability	NFR-U1	GUI must be user-friendly , supporting clear dashboard displays and drag-and-drop file upload.	Direct necessity for reducing the training time and cognitive burden on security analysts. Achieved through the highly structured, modern design implemented via CustomTkinter.

2.3 Current System

The current paradigm for security monitoring relies on two inadequate strategies: **Manual Log Analysis** and **Basic SIEM / Rule-Based Tools**.

2.3.1 Manual Log Analysis: Inherently Unscalable and Prone to Error

Manual log analysis is fundamentally unscalable and introduces significant security and operational risks. Security analysts often depend on command-line tools such as *grep*, *awk*, or manually crafted regular expressions to filter and interpret large volumes of log data. This workflow is not only time-consuming and error-prone but also requires a level of technical proficiency that many system administrators may not possess.

Because the analysis is performed in isolated terminal windows, insights remain fragmented and are not automatically shared, visualized, or contextualized. Analysts must manually piece together log fragments to understand system behaviors, which prevents comprehensive, end-to-end evaluation of security incidents. This fragmented process makes **temporal correlation** extremely difficult, meaning analysts cannot easily reconstruct the sequence of events in an

attack chain. As a result, early indicators of compromise are often overlooked, delays occur in incident response, and organizations remain vulnerable to advanced threats.

2.3.2 Basic SIEM / Rule-Based Tools: Technologically Rigid and Operationally Noisy

These legacy solutions fail against contemporary threats due to their dependency on outdated technological stacks. Their rigidity makes them immediately ineffective against zero-day exploits because they require a human to recognize the threat, write a new rule, and deploy it—a time-consuming process. Their inability to comprehend contextual normalcy results in an excessive volume of false positive alerts (operational noise), generating a dangerous cycle of analyst burnout and diminishing the perceived trustworthiness of the alert system. AI Log Guard provides a proactive, high-value, and cost-effective replacement by employing an adaptive AI core that learns expected system behavior. The integration of AI shifts the security paradigm from defending against the known bad to detecting any meaningful deviation from the known good.

trustworthiness of the alert system. **AI Log Guard** provides a proactive, high-value, and cost-effective replacement by employing an adaptive AI core that learns expected system behavior.

Chapter 3

Requirements Analysis

3. Requirement Analysis

This chapter formalizes the functional and non-functional requirements of the **AI LogGuard** system based on the project proposal, Software Requirement Specification (SRS), and implementation details. It utilizes Use Case Diagrams and tables to model user interactions and provides a comprehensive breakdown of system capabilities

3.1 Use Cases Diagrams

The Use Case diagram provides a high-level overview of the system's functionality and the actors who interact with it. For **AI LogGuard**, the primary actor is the **Security Analyst (User)**, who interacts with various modules of the application.

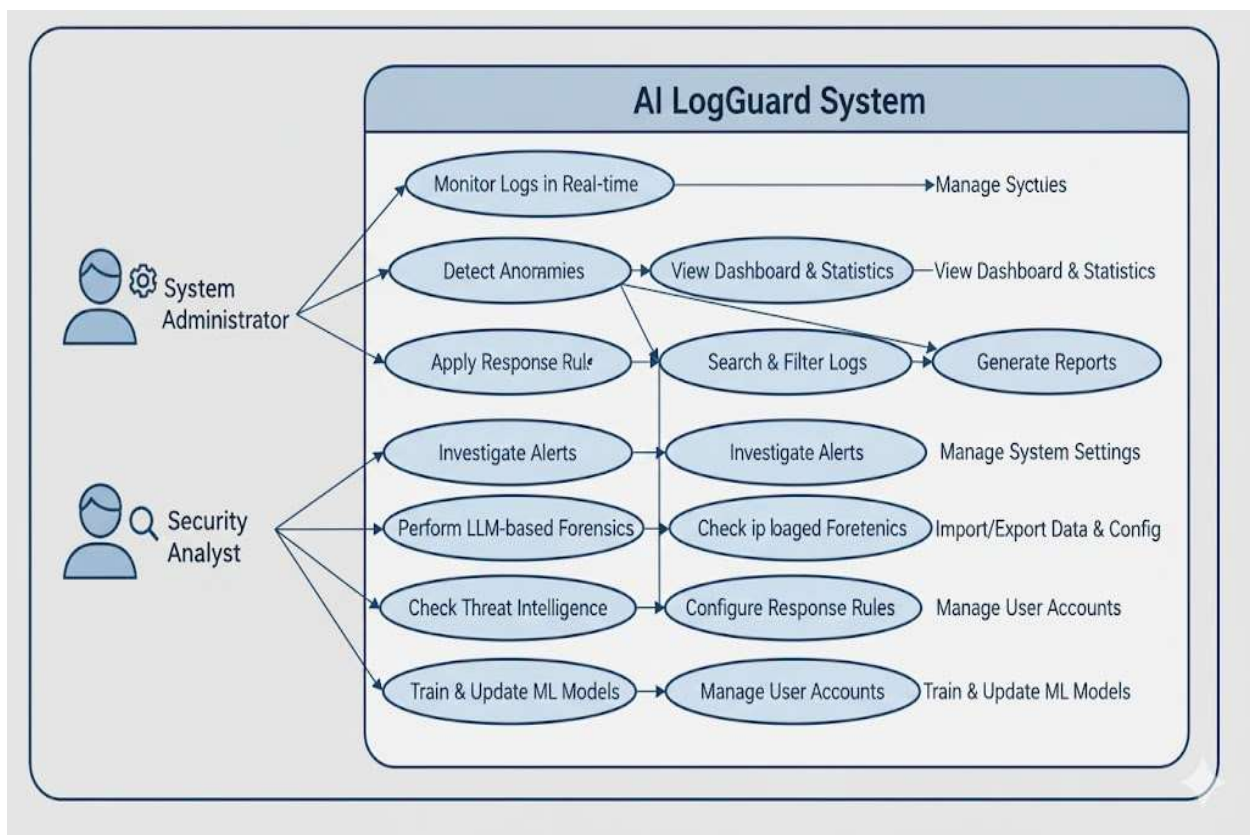


Figure 3.1 Ai LogGuard Use Case Diagram

3.1.1 Actors:

- **Security Analyst / Administrator:** The primary user responsible for uploading logs, monitoring real-time traffic, configuring detection rules, and reviewing forensic reports.
- **AI Engine (System Actor):** The internal component (Isolation Forest & LLM) that processes data and triggers alerts autonomously.

3.1.2 Main Use Case:

1. User Authentication & Session Management

- Secure login and logout functionality for authorized users.
- Role-based access control (Admin, Analyst, Viewer).
- Session handling with automatic timeout to prevent unauthorized access.
- Persistent user profiles and activity logging for audit trails.

2. Live Monitoring of System Logs

- Real-time viewing of incoming log streams (system, network, firewall, application logs).
- Visual dashboards showing active connections, anomalies, alerts, and severity levels.
- Simulated traffic mode for demonstration or testing of detection pipelines.
- Automatic alerts (sound, popup, or visual highlight) when suspicious behavior occurs.

3. Forensic Log Analysis using LLM

- Deep inspection of selected log entries or log segments.
- Natural-language explanations generated by the integrated LLM (e.g., “What does this log mean?”).
- Pattern recognition for malicious behaviours such as brute force attempts, port scanning, privilege escalation, etc.
- Step-by-step reasoning to help analysts understand attack flow and event timelines.

4. Rule Management & Response Automation

- Create, edit, and manage static security rules (e.g., “Block IP on 5 failed logins”).
- Threshold-based anomaly rules to trigger automated actions.
- Blacklisting/whitelisting of IPs, usernames, or host machines.
- Integration with log pipelines for real-time enforcement of rules.

5. Reporting & Historical Analysis

- Generate comprehensive PDF or CSV reports on anomalies, alerts, or user activity.
- Summaries of attack patterns, severity trends, and system health over time.
- Exportable daily, weekly, or monthly incident reports.
- Visual analytics: charts, heat maps, timeline graphs, and frequency distributions.

6. Threat Intelligence Integration

- Real-time lookup of IP reputation using external threat intelligence feeds.
- Geolocation of suspicious IPs (country, city, ASN, ISP).
- Enrich log events with threat score, known malicious behavior, and blacklist status.
- Helps analysts quickly identify whether an IP is benign or part of a known attack network.

3.2 Detailed Use Case

This section provides a tabular description of the critical use cases implemented in the system.

3.2.1 Use Case 1: Monitor Logs in Real-Time

Table 3.1 Monitor Logs in Real-Time

Attribute	Description
Use Case ID	UC-01
Use Case Name	Monitor Logs in Real-Time
Primary Actor	Security Analyst
Description	The user initiates the live monitoring dashboard to visualize log traffic and detect anomalies as they occur.

Pre-conditions	• User is logged in. • Database connection is established. • Core Logic thread is initialized.
Post-conditions	Real-time graphs are updated, and anomalies are logged to the database.
Normal Flow	• User navigates to "Live Monitor" page. • User selects source (File or Simulation) and clicks "Start". • System reads log streams via core_logic.py. • System calculates anomaly scores using Isolation Forest. • UI updates the "Events/Sec" graph and Log Console.
Alternative Flow	• If the log file path is invalid, the system displays an error toast notification. • If the AI model is not loaded, the system reverts to heuristic detection.

3.2.2 Use Case 2: AI Forensic Analysis (LLM Integration)

Table 3.2 AI Forensic Analysis (LLM Integration)

Attribute	Description
Use Case ID	UC-02
Use Case Name	Generate LLM Forensic Report
Primary Actor	Security Analyst
Description	The user requests a detailed natural language explanation for a specific suspicious log entry.
Pre-conditions	• User has selected a log entry from the Alerts page. • 2. Valid LLM API Key is configured in config.py.
Post-conditions	A structured analysis (Summary, Severity, Recommendation) is saved to the database.

Normal Flow	• User selects an alert and clicks "Analyze (LLM)". • System sends the log snippet to llm_service.py. • External API (OpenRouter) processes the request. • System parses the JSON response. • 5. Result is displayed in the "Investigation Panel".
Exceptions	API Timeout or Quota Exceeded: System returns a fallback error message to the UI.

3.2.3 Use Case 3: Configure Response Rules

Table 3.3 Configure Response Rules

Attribute	Description
Use Case ID	UC-03
Use Case Name	Manage Response Rules
Primary Actor	Administrator
Description	The user defines static rules (IF condition THEN action) to handle known threats automatically.
Normal Flow	• User navigates to "Response Rules" page. • User clicks "Add New Rule". • User defines condition (e.g., "Severity = Critical"). • User defines action (e.g., "Block IP"). • User clicks "Save". • 6. System persists rule to SQLite and reloads the engine.

3.3 Functional Requirements

These requirements define the specific behaviors and functions the system must support, directly derived from the source code implementation.

FR-01: Log File Submission & Ingestion

- **Description:** The system shall allow users to input log data via local file selection or simulated network streams.
- **Implementation:** LiveMonitorPage.py provides a file picker dialog and inputs for stream URLs. core_logic.py contains _file_loop and _sim_loop methods to handle data ingestion.

FR-02: Log Preprocessing & Parsing

- **Description:** The system shall parse raw log lines to extract key metadata: Timestamp, Severity Level, IP Address, and Message Content.
- **Implementation:** core_logic.py uses Regular Expressions (IP_REGEX) and string parsing logic within _process_single_log to structure raw text.

FR-03: Feature Extraction

- **Description:** The system shall convert text-based log messages into numerical feature vectors suitable for machine learning analysis.
- **Implementation:** core_logic.py utilizes TfidfVectorizer (from Scikit-learn) to transform log messages into TF-IDF vectors.

FR-04: Anomaly Detection (AI Analysis)

- **Description:** The system shall use a pre-trained Machine Learning model to classify log entries as "Normal" or "Anomalous" in real-time.
- **Implementation:** core_logic.py employs the Isolation Forest algorithm (self.model.predict) to assign anomaly scores.

FR-05: Real-Time Alerts

- **Description:** The system shall immediately notify the user upon detecting a high-severity anomaly or matching a response rule.

- **Implementation:** AlertsPage.py displays a live feed of alerts. The NotificationDropdown class in AiLogGuard.py provides quick access to recent notifications.

FR-06: Automated Threat Response

- **Description:** The system shall execute predefined actions (e.g., Block IP, Send Email) when specific threat conditions are met.
- **Implementation:** ResponseRulesPage.py allows users to define rules. core_logic.py executes these rules via check_and_execute_rules, updating the active_blocklist.

FR-07: AI Forensic Analysis (LLM Integration)

- **Description:** The system shall allow users to send suspicious log snippets to an external Large Language Model (LLM) for detailed analysis and remediation advice.
- **Implementation:** LLMForensicsPage.py captures user input. llm_service.py sends the data to the OpenRouter API and parses the structured JSON response.

FR-08: Report Generation

- **Description:** The system shall generate downloadable reports (PDF, CSV) summarizing log activities, anomalies, and audit history.
- **Implementation:** ReportsPage.py handles data querying from database_manager.py and formats the output using csv and reportlab libraries.

FR-09: System Configuration & User Management

- **Description:** The system shall allow administrators to manage user accounts (create/delete), update passwords, and configure application settings (theme, notifications).
- **Implementation:** SettingsPage.py provides the UI for these actions, backed by database_manager.py methods like create_user and set_setting.

3.4 Non-Functional Requirements

These requirements specify the criteria that can be used to judge the operation of the system, focusing on quality attributes.

NFR-01: Performance & Latency

- **Requirement:** The system must process individual log lines and update the UI with minimal latency (target < 100ms per line) to ensure a "real-time" experience.
- **Constraint:** The LiveMonitorPage graph and log feed must remain responsive even under high data load.
- **Implementation:** Multi-threading is used in AiLogGuard.py and core_logic.py to separate the UI thread from the heavy processing logic.

NFR-02: Scalability

- **Requirement:** The system architecture should support the addition of new modules (e.g., new AI models or report types) without rewriting the core engine.
- **Constraint:** The database and in-memory queues (deque) must handle thousands of log entries without crashing.
- **Implementation:** Modular design (Page classes, separate Database Manager) and configurable queue sizes (maxlen=1000) ensure scalability.

NFR-03: Usability

- **Requirement:** The User Interface must be intuitive, with a consistent modern design (Dark Mode), clear navigation, and responsive elements.
- **Constraint:** The application must scale correctly across different window sizes and resolutions.
- **Implementation:** Built using CustomTkinter with a defined theme (config.py) and responsive grid layouts.

NFR-04: Reliability & Fault Tolerance

- **Requirement:** The system must handle errors gracefully (e.g., network failure during LLM analysis or missing database file) without crashing.
- **Constraint:** Background threads must be managed safely to prevent zombie processes upon application exit.
- **Implementation:** Extensive try-except blocks in `core_logic.py` and `database_manager.py`. Thread management via `stop_threads` methods in all Page classes.

NFR-05: Security

- **Requirement:** Sensitive user data (passwords) must be hashed before storage. API keys for external services must be handled securely.
- **Constraint:** The system must prevent SQL injection attacks.
- **Implementation:** `database_manager.py` uses `hashlib.pbkdf2_hmac` for password hashing and parameterized SQL queries (`% placeholders`) to prevent injection.

NFR-06: Compatibility

- **Requirement:** The application should be runnable on major desktop operating systems (Windows, Linux, macOS).
- **Implementation:** Written in cross-platform **Python** using standard libraries and `CustomTkinter`, which supports all major OSs.

Chepter 4

Design and Architecture

4. Design and Architecture

This chapter provides a comprehensive overview of the architectural design and modular composition of the AI LogGuard system. It explains how the system is structured, how its components interact, and how this structure enables intelligent, scalable, and efficient security monitoring. The design follows modern engineering principles to ensure extensibility, modularity, and robustness during real-time log analysis.

4.1 System Architecture

The AI Log Guard employs a Modular Microservices-inspired Architecture, implemented as a high-performance Python application. The system is logically separated into four distinct layers: the Presentation Layer (GUI), the Business Logic Layer (Core Engine), the Persistence Layer (Database), and the External Integration Layer (APIs). This decoupling ensures scalability, maintainability, and adherence to the non-functional requirement of modularity (NFR-C1).

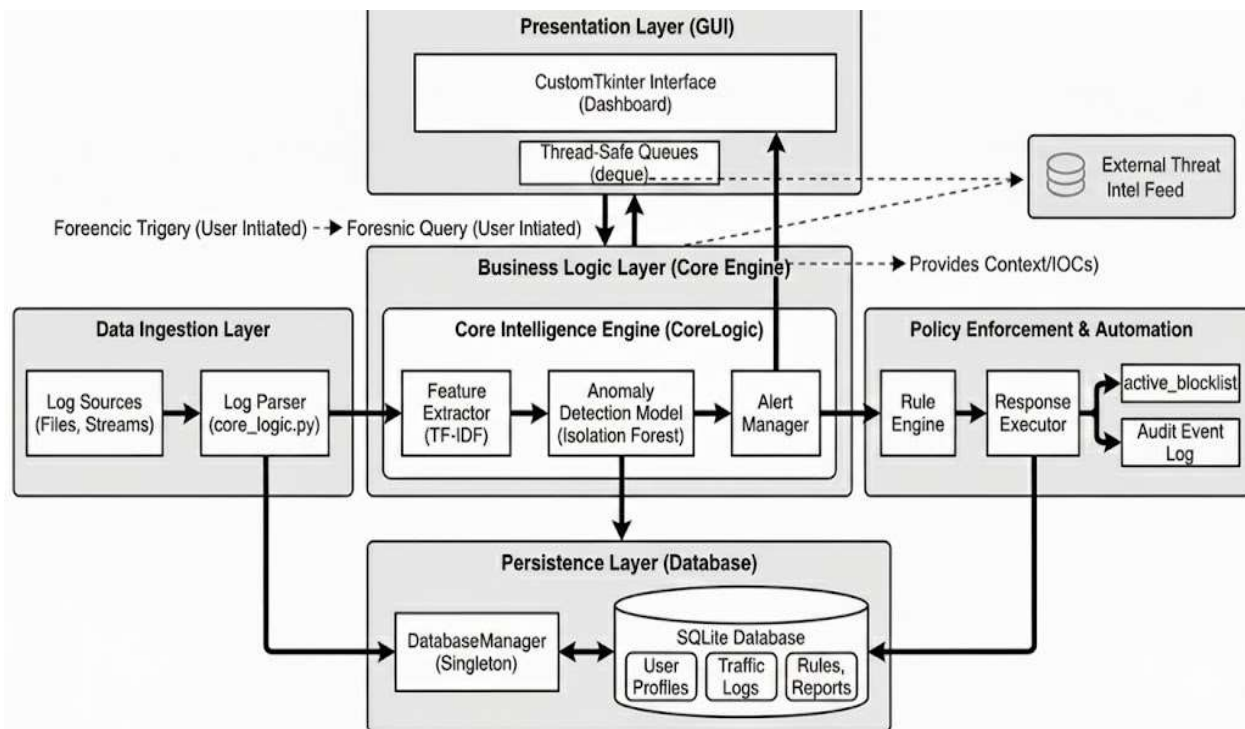


Figure 4.1 AI LogGuard Architecture

The data flow is unidirectional from **Log Sources** to the **Log Database**, with the **Anomaly Detection Engine** acting as the central intelligence broker, feeding results to the **Alert Manager** and **User Interface**.

4.1.1 Architectural Components:

The AI LogGuard system is composed of several well-defined architectural components, each fulfilling a specific role within the multi-layered architecture. These components work together to enable efficient log ingestion, intelligent anomaly detection, automated rule enforcement, secure data persistence, and an interactive user interface.

4.1.1.1 Data Ingestion Layer

i. Log Sources

- Serves as the primary entry point for system and network data.
- Supports ingestion from multiple input formats including **local log files** (.log, .txt, .csv) and **real-time simulated streams** provided by LiveMonitorPage.py.
- Designed to scale with future integrations such as Syslog, Windows Event Viewer, or remote log agents.

ii. Log Parser

- Implemented within core_logic.py.
- Responsible for transforming raw, unstructured text into structured log records.
- Performs:
 - Cleaning and normalization of raw text.
 - Removal of ANSI escape sequences (console color codes).
 - Extraction of metadata such as timestamps, IP addresses, and log levels using advanced Regular Expressions including IP_REGEX.
- Ensures that downstream modules receive clean, uniform, machine-processable events.

4.1.1.2 Core Intelligence Engine (CoreLogic)

i. Feature Extractor

- Converts parsed log messages into numerical feature vectors.
- Uses **TF-IDF (Term Frequency–Inverse Document Frequency)** for text vectorization.
- Captures the semantic patterns and frequency metrics needed for statistical anomaly detection.

ii. Anomaly Detection Model

- Implements **Isolation Forest**, a widely used unsupervised machine learning algorithm for anomaly scoring.
- Assigns each log entry an **anomaly score** based on deviation from learned patterns.
- Operates continuously during both historical analysis and real-time monitoring.

ii. Alert Manager

- Evaluates anomaly scores against configurable thresholds.
- When a suspicious entry is detected, the Alert Manager pushes a structured alert object into the **global_alert_queue** (thread-safe).
- Supports severity levels (Low, Medium, High, Critical) for the GUI and reporting modules.

4.1.1.3 Policy Enforcement & Automation

i. Rule Engine

- Inspects anomaly events and compare them against user-defined **Response Rules**.
- Example: *“Block IP if 5 consecutive failed logins occur within 2 minutes.”*
- Rules can be added, modified, or removed through the GUI.

ii. Response Executor

- Executes automated responses triggered by matched rules.
- Supports actions such as:
 - Adding an IP to the **active_blocklist**.
 - Logging a **security audit event**.
 - Triggering custom workflow actions (future extension).
- Ensures the system not only detects threats but can also take preventive measures.

4.1.1.4 Persistence Layer

1i. SQLite Database

- Managed through the DatabaseManager singleton class.
- Ensures **thread-safe** read/write operations while multiple modules operate concurrently.
- Stores:
 - User Profiles & Authentication Data
 - Historical Traffic Logs
 - Rule Configurations
 - Reports & Analysis Summaries
- Lightweight, reliable, and suitable for local deployments without requiring external DB servers.

4.1.1.5 Presentation Layer (GUI)

ii. CustomTkinter User Interface

- Provides a modern, responsive desktop GUI for analysts and administrators.
- Runs on the main application thread while heavy tasks are offloaded to background worker threads.

- Utilizes **thread-safe queues (deque)** to receive events and alerts from the Core Engine without freezing the interface.
- Includes modules for:
 - Live Monitoring
 - Forensic Log Inspection
 - Rule Management
 - Report Generation
 - Threat Intelligence Lookup

4.2 Data Representation

Data integrity and persistence are managed by **SQLite** via the **DatabaseManager** class. Key data structures include:

- **Users:** Stores authentication details (hashed passwords, salt) and user metadata.
- **Traffic Logs:** The main data table, storing every ingested log line with extracted metadata (timestamp, source_ip, level, message, category). This supports the **Live Monitor**.
- **Response Rules:** Stores the JSON-formatted trigger **Condition** and **Action** for automated responses. Sorted by priority (an integer field).
- **Forensic Analysis:** Stores the structured Pydantic output from the LLM, including analysis_summary, recommendation, and a log_snippet for historical review.
- **IP Reputation:** A cache table storing geo-location data (lat, lon, country, isp) for IPs to minimize API calls, used by the **Threat Intel** page.
- **Audit Log (audit_log):** Records administrative actions (e.g., "User Login", "Rule Created") for security compliance.
- **Settings (settings):** A key-value store for application configuration (e.g., theme_mode, ui_scale).

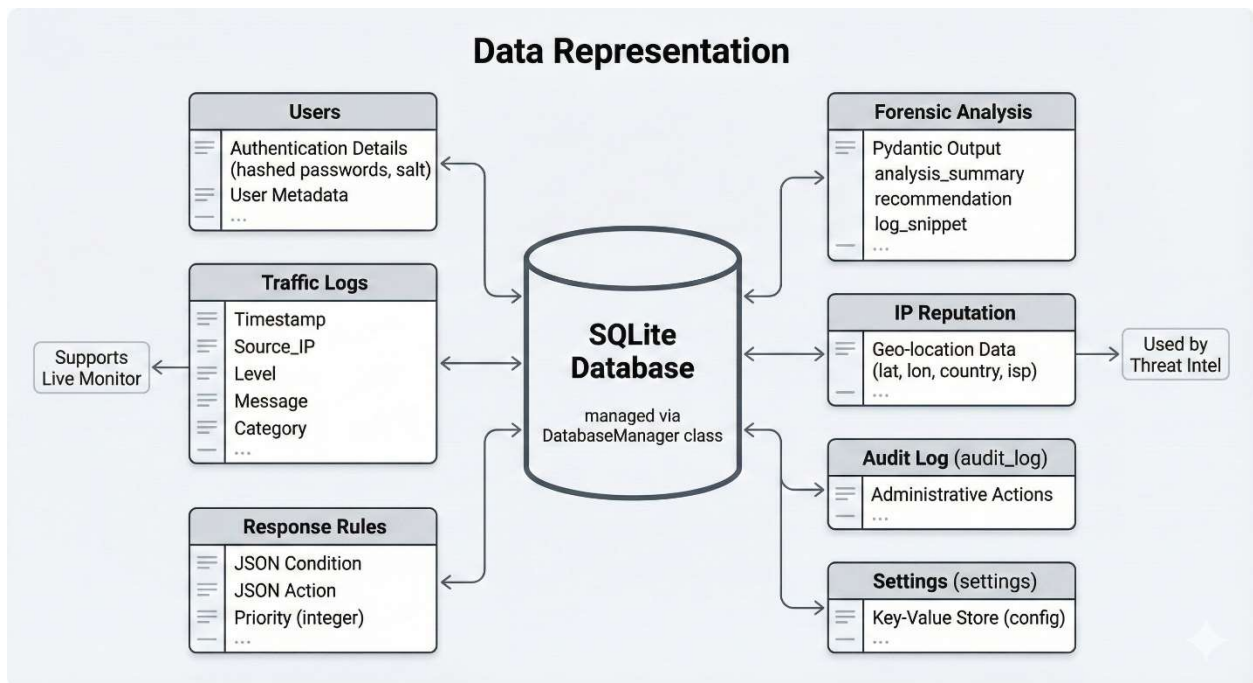


Figure 4.2 AI LogGuard Data Representation

4.3 Process Flow/Representation

The process flow visualizes the mandatory pipeline for every ingested log line.

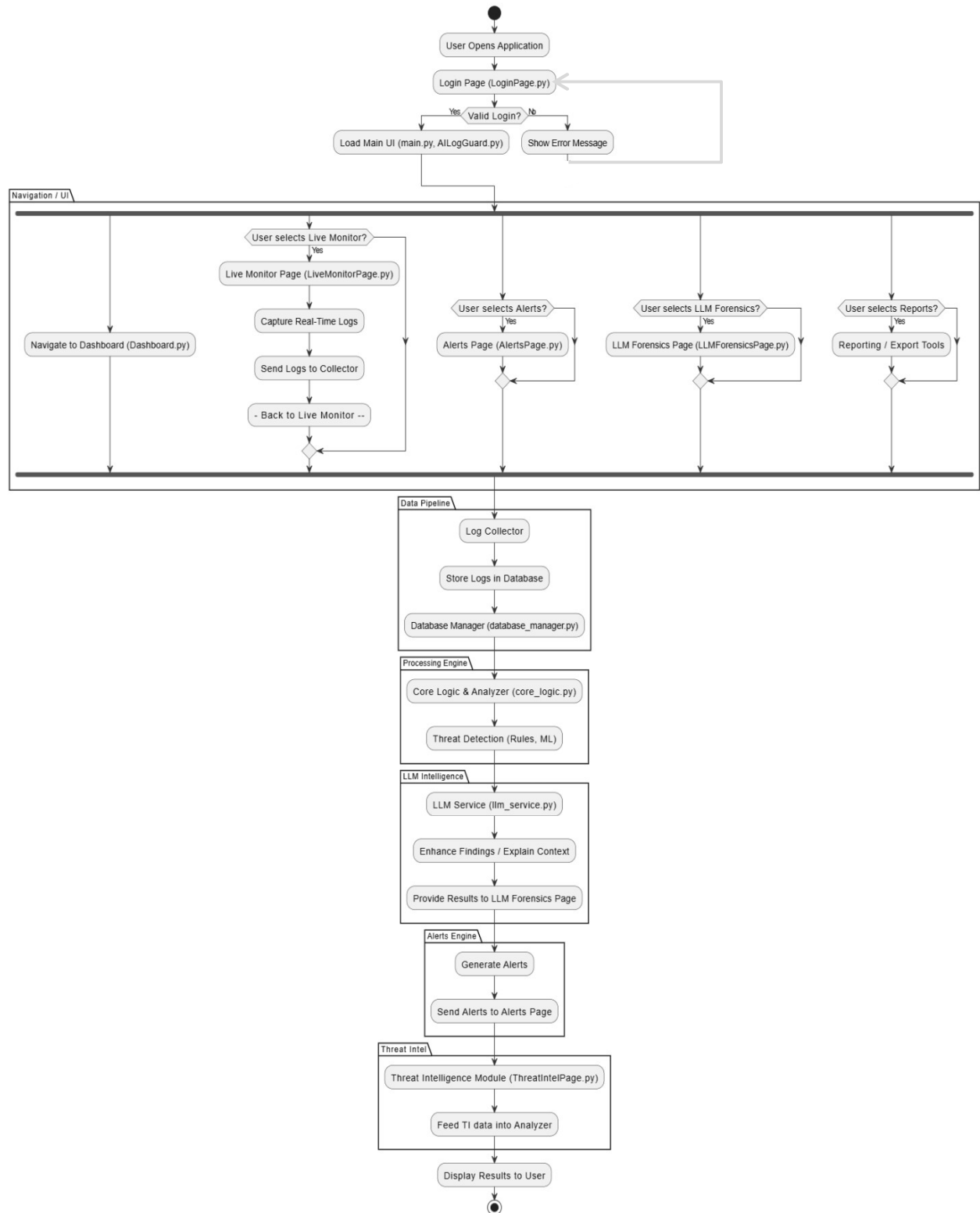


Figure 4.3 AI LogGuard Process Flow Diagram

This diagram is a UML Activity Diagram that illustrates the end-to-end process flow of an "AI Log Guard" application. It details both user interaction and the underlying system processes, from application startup to displaying final results.

Overall Purpose:

The diagram maps out the activities involved in a log monitoring and analysis system, emphasizing the interaction between user interface components and backend processing. It shows how logs are collected, processed by various engines (including an LLM for AI intelligence), and ultimately presented to the user through different interfaces.

4.3.1.1 Key Sections and Their Descriptions:

i. Application Startup and Login:

- The flow begins with a user opening the application.
- They are presented with a login page (LoginPage.py).
- A decision point checks for a "Valid Login?".
- If "Yes," the main UI (main.py, AILogGuard.py) is loaded.
- If "No," an "Show Error Message" is displayed, and the process stops.

ii. Navigation / UI (User Interface Activities):

- This section represents the user's interaction with the application's various views and features. It uses a fork to indicate that the user can navigate to different sections, though the diagram shows a sequential flow for clarity within this context.
- Navigate to Dashboard (Dashboard.py): The default or primary view for analytics.
- User selects Live Monitor?
- If "Yes": Leads to the "Live Monitor Page (LiveMonitorPage.py)" which "Capture Real-Time Logs" and "Send Logs to Collector." The " -- Back to Live Monitor --" indicates a continuous or iterative process here.
- User selects Alerts?
- If "Yes": Leads to the "Alerts Page (AlertsPage.py)" where alerts are displayed.
- User selects LLM Forensics?

- If "Yes": Leads to the "LLM Forensics Page (LLMForensicsPage.py)" to view AI-driven insights.
- User selects Reports?
- If "Yes": Leads to "Reporting / Export Tools" for generating summaries.

iii. Data Pipeline:

- This segment focuses on the initial handling of log data.
- Log Collector: Responsible for receiving and aggregating logs.
- Store Logs in Database: Persists the collected log entries.
- Database Manager (database_manager.py): The component managing the database operations.

iv. Processing Engine:

- This section describes the core logic for analyzing logs.
- Core Logic & Analyzer (core_logic.py): The central component that parses logs, classifies events, and applies initial rules.
- Threat Detection (Rules, ML): This sub-activity within the processing engine identifies potential threats based on predefined rules and machine learning models.

v. LLM Intelligence:

- This highlights the role of Artificial Intelligence, specifically a Large Language Model.
- LLM Service (llm_service.py): The dedicated service for AI-driven analysis.
- Enhance Findings / Explain Context: The LLM provides deeper insights, context, and explanations for suspicious patterns.
- Provide Results to LLM Forensics Page: Sends the AI-generated forensic details back to the UI for viewing.

vi. Alerts Engine:

- Focuses on the generation and delivery of alerts.
- Generate Alerts: Creates actionable alerts based on detected threats or anomalies.
- Send Alerts to Alerts Page: Delivers these alerts to the dedicated Alerts Page in the UI.

vii. Threat Intel:

- Integrates external or internal threat intelligence.

- Threat Intelligence Module (ThreatIntelPage.py): Manages threat intelligence data.
- Feed TI data into Analyzer: Incorporates threat intelligence to enrich log analysis and improve detection.

viii. Display Results to User:

- The final activity, where all processed information, alerts, and insights are made available for the user to view.

4.4 Design Models

4.4.1 Sequence Diagram

The Sequence Diagram details the execution order for a core process, such as submitting a log line for analysis and alerting.

4.4.1.1 Sequential Steps for Anomaly Detection:

- **User** initiates monitoring (e.g., uploading a file).
- **CoreLogic** reads a log line and sends it to the **Log Parser**.
- **Feature Extractor** generates the feature vector.
- **AI Model** executes prediction and returns the score.
- If anomalous, **CoreLogic** pushes data to **Alert Manager** and **Dashboard/UI**.
- **Alert Manager** checks **Response Rules** and executes actions (e.g., **Block IP**).

4.4.2 Scenario 1: Real-Time Anomaly Detection & Alerting

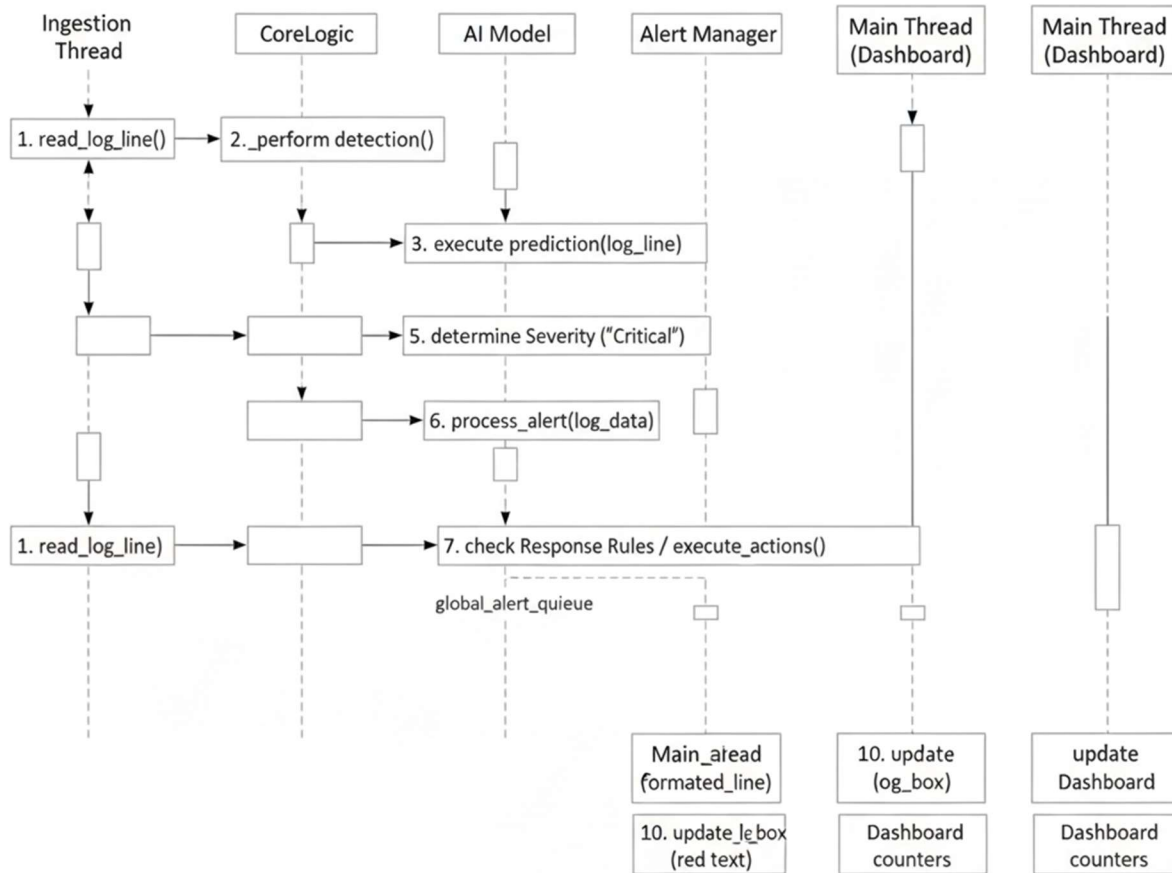


Figure 4.3 Anomaly Detection

- **Ingestion Thread** reads a log line.
- Calls CoreLogic._perform_detection().
- **AI Model** returns Anomaly Score.
- CoreLogic determines Severity = "Critical".
- CoreLogic adds alert to global_alert_queue.
- CoreLogic calls LiveMonitorPage.process_line().

- **Main Thread** updates log_box with red text.
- **Main Thread** updates Dashboard counters.

4.4.3 Scenario 2: LLM Forensic Analysis

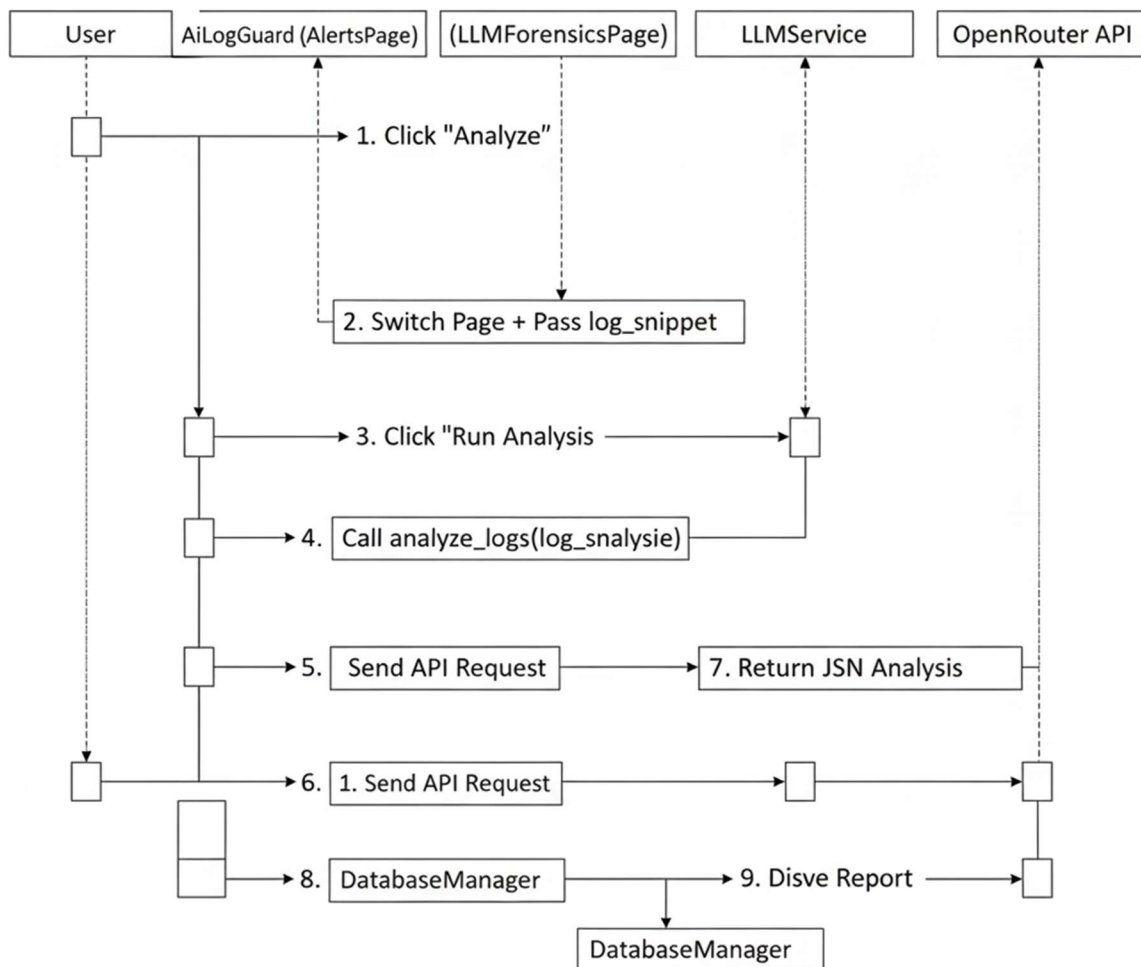


Figure 4.4 LLM Forensics

- User clicks "Analyze" on AlertsPage.
- AiLogGuard switches to LLMForensicsPage and passes the log snippet.
- User clicks "Run Analysis".
- LLMForensicsPage calls llm_service.analyze_logs().
- llm_service sends request to **OpenRouter API**.

Chapter 5

Implementation

5. Implementation

This chapter details the technical execution of the project, focusing on the core intelligence and the specific technologies used to build the user interface and external integrations.

5.1 Algorithm

The intelligence pipeline is governed by two algorithms: feature engineering and the detection model.

5.1.1 Log Parsing and Preprocessing

Implemented in `core_logic.py`, the system uses robust **Regular Expressions (RegEx)** to normalize unstructured data.

- **IP Extraction:** The regex `r'(?:\d{1,3}\.){3}\d{1,3}'` identifies IPv4 addresses within log lines.
- **Color Stripping:** ANSI escape codes (often found in raw console logs) are stripped using regex to ensure clean text for the ML model.
- **Timestamp Normalization:** Logs are parsed to standard Python datetime objects to allow for time-based filtering and "Repeated Event" detection.

5.1.2 Feature Engineering (TF-IDF Vectorization)

- **Goal:** Convert textual log data into a numerical vector while emphasizing terms that are unique and potentially malicious (e.g., "UNION SELECT," "failed_login").
- **Implementation:** `TfidfVectorizer` (Scikit-learn) is trained on a large corpus of normal and simulated attack logs. In the `_perform_detection` function of `core_logic.py`, the `vectorizer.transform([text])` method converts the current log message into the feature matrix used by the AI model.

5.1.3 Anomaly Detection (Isolation Forest)

- **Goal:** Quickly and accurately identify multivariate outliers in the feature space.

- **Implementation:** The **Isolation Forest** model is chosen for its speed (ideal for NFR-P1). In `core_logic.py`, the heuristic score is boosted (score += 1.5) when the ML prediction (`model.predict(vec)`) returns an anomaly flag (-1). This combines heuristic speed with AI intelligence.

```
# Snippet from core_logic.py -> _perform_detection
# ... Heuristic scoring increases score based on keywords ...

# --- AI/ML Scoring ---
if SKLEARN_AVAILABLE and self.model and self.vectorizer:
    try:
        # Vectorize the text part of the log message
        text_part = line.split(" ", 2)[-1].strip()
        vec = self.vectorizer.transform([text_part])

        # Predict: -1 indicates an anomaly
        if int(self.model.predict(vec)[0]) == -1:
            ai_flag = True
            ai_score = 0.9
            score += 1.5 # Boost the overall severity score
            if category == "General": category = "ML Anomaly"
    except Exception as e:
        print(f"AI Prediction failed (ML): {e}")

# ... Final Severity Mapping ...
```

5.1.4 Automated Response Rule Engine

The **Automated Response Rule Engine** is a core security component implemented in `core_logic.py` within the function `_check_and_execute_rules`. It provides intelligent, automated

decision-making capabilities by continuously evaluating incoming anomaly events against user-defined security rules.

i. Logic

- The Rule Engine dynamically loads all **enabled response rules** from the database at runtime.
- Each rule contains attributes such as **priority**, **condition parameters**, **action type**, and **activation thresholds**.
- Rules are evaluated in **priority order**, ensuring that high-severity or critical rules execute before general-purpose rules.
- The engine is designed for extensibility, allowing new rule types and conditions to be added without modifying the core infrastructure.

ii. Conditions

The engine supports a range of conditional checks to determine whether a rule should be triggered:

- **Severity-Based Checks**

Evaluates whether the anomaly severity meets the rule's configured threshold (e.g., High, Critical).

- **IP-Based Matching**

Matches the source IP of an event with rule criteria, allowing actions to be targeted at specific IPs or ranges.

- **Time-Window Frequency Evaluation**

Tracks the number of occurrences of a specific anomaly type within a defined time window.

- Example: Detect **"5 failed login attempts within 60 seconds"**, a common brute-force signature.
- Utilizes timestamped event logs to ensure accurate and efficient frequency counting.

- **Pattern or Keyword Matching** (*extendable*)

Allows rules to trigger based on specific log patterns or text fragments.

iii. Actions

Upon condition satisfaction, the Rule Engine executes the configured automated response.

Supported action types include:

- **Block IP Address**
 - Adds the offending IP to the **active_blocklist**, preventing future events from that source.
 - Logged as an audit event for traceability.
- **Send Email Alert (Simulated)**
 - Generates a simulated email notification to the administrator.
 - Useful for integration with real email providers in future expansions.
- **Execute External Script**
 - Allows running predefined scripts for custom responses such as:
 - Firewall rule updates
 - System commands
 - External API calls
 - Enhances flexibility and allows organizations to embed their own security actions.

Each executed action is logged in the system's audit trail for compliance and post-incident analysis.

5.2 External APIs

The system uses three primary external services for intelligence and functionality.

Table 5.1 External APIs

Name of API	Description of API	Purpose of usage	Function/Class Name in which it is used
LLM Service (OpenRouter/ Gemini)	Cloud-based LLM (e.g., Google's Gemini-2.0-Flash) for structured natural language analysis of complex log events.	FR-06 (LLM Forensic Analysis): Provides detailed summaries, observations, and remediation steps.	STRESSED.analyze_logs() (llm_service.py), LLMForensicsPage
IP-API.com	A public geo-location API providing IP metadata (country, lat/lon, ISP, proxy status).	Threat Intelligence: Geocodes attacking IPs for visualization on the Global Threat Map (ThreatIntelPage) and enhances response rules.	_ip_worker_loop (ThreatIntelPage.py)
ReportLab	Python library for programmatically generating PDF documents.	FR-07 (Reporting): Generates structured, printable PDF audit reports from historical log data.	_save_pdf_report () (ReportsPage.py)

5.3 User Interface

The application interface is built using **CustomTkinter**, providing a modern, dark-themed, and responsive GUI (NFR-U1, NFR-C2).

5.3.1 Component Implementation

- **UI Framework:** **CustomTkinter** for high-DPI scaling and themed widgets.
- **Aesthetics:** Styling is controlled centrally via config.py (e.g., COLOR_PRIMARY, COLOR_CARD), ensuring a consistent and professional look (modern_components.py).

5.3.2 Key Page Interfaces

5.3.2.1 Dashboard (Dashboard.py)

The central command center featuring:

- **Efficacy Graph:** A real-time Matplotlib line chart showing "Detected Anomalies" vs. "Blocked Threats" over time.
- **Stats Cards:** Live counters for total anomalies and logs processed.
- **Global Threat Map:** A visualization using Cartopy (or scatter plot fallback) to plot the geo-location of attacking IPs based on data from IP-API.com.

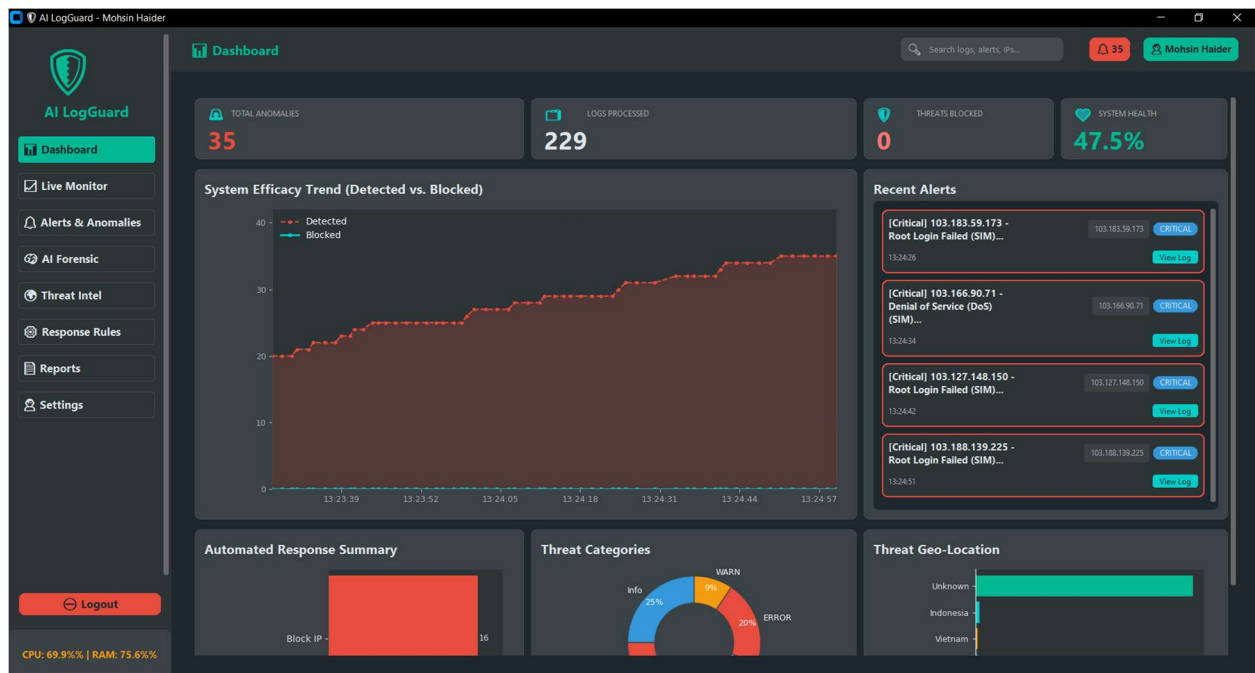


Figure 5.1: Dashboard Interface

5.3.2.2 Live Monitor (LiveMonitorPage.py)

A dedicated console for real-time log observation.

- **Log Stream:** A high-performance text widget that color-codes logs by severity (Red for Critical, Orange for Warn).

- **Architecture:** It uses a decoupled UI update loop (`_start_ui_loop`) that consumes data from a shared queue. This ensures the interface never freezes, even when processing thousands of logs per second.

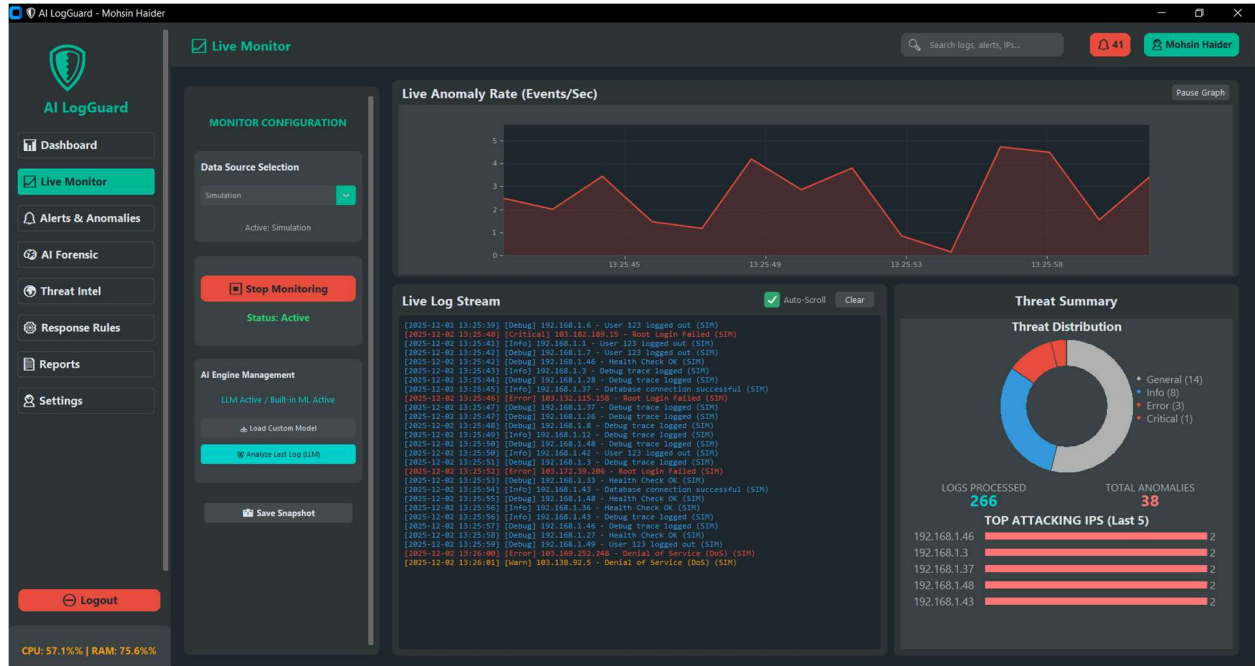


Figure 5.2: Live Monitor Interface

5.3.2.3 Response Rules Editor (ResponseRulesPage.py)

A graphical interface for managing security policies.

- **Dynamic Forms:** Depending on the selected Condition (e.g., "Repeated Event"), the UI dynamically renders the appropriate input fields (e.g., "Time Window", "Count").
- **Abstraction:** It abstracts the complex underlying JSON data structure of rules, allowing non-technical users to define sophisticated logic like "Block IP if Severity is Critical".

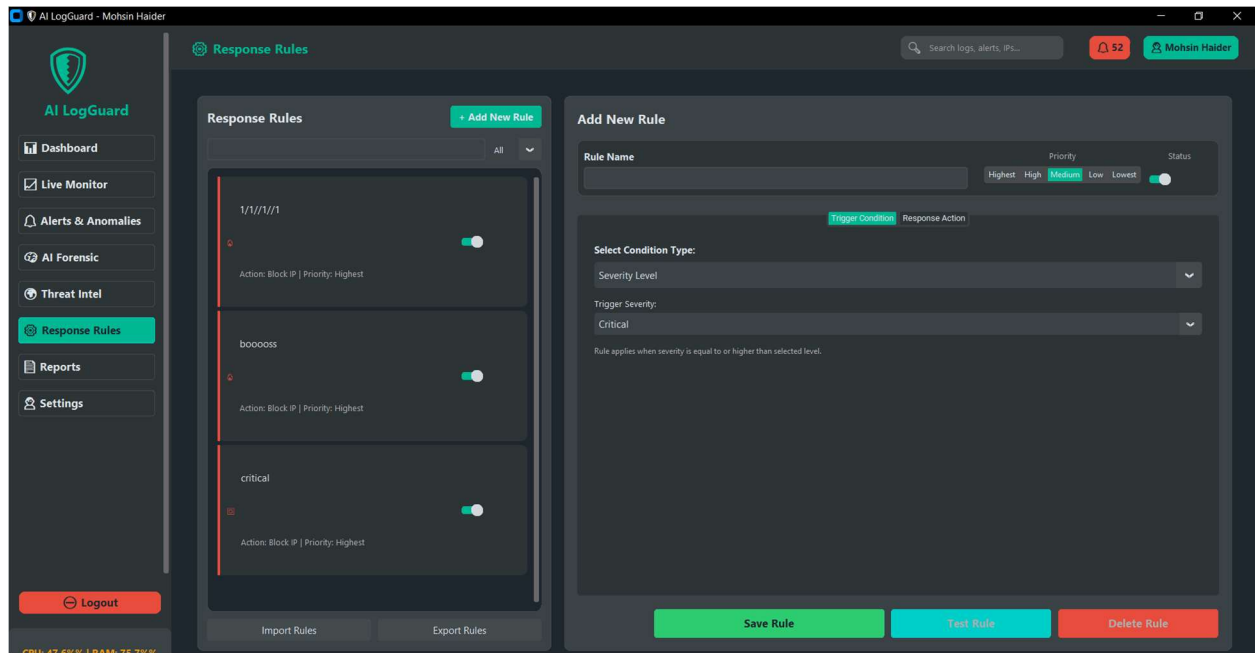


Figure 5.3: Response Rules Editor Interface

5.3.2.4 AI Forensics (LLMForensicsPage.py)

An interface for the AI analyst.

- **Workflow:** Users can select a log source (DB or File), define a scope, and submit it for analysis.
- **Output:** The page renders the LLM's JSON response into a readable report with separate tabs for "Summary", "Events", and "Traffic Patterns", providing a clear narrative of the security incident.

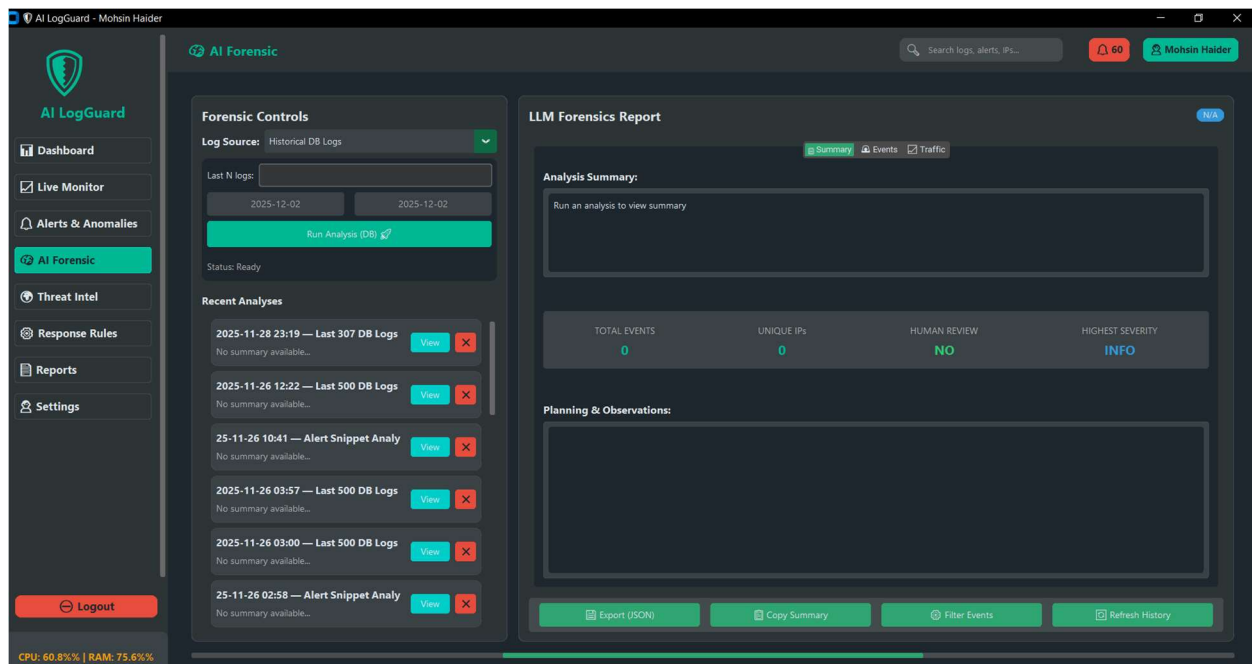


Figure 5.4: AI Forensics interface

5.3.2.5 Alerts & Anomalies (AlertsAnomaliesPage.py)

The primary interface for reviewing and triaging security events identified by the system's detection engine.

- **Alert List:** A filterable and searchable table displaying all generated security alerts. Each entry includes the Timestamp, Severity (Critical, High, Medium, Low), Rule/Model Triggered, and Target IP/Entity.
- **Anomaly Score:** Visual indicators (e.g., bar charts or heatmaps) showing the statistical deviation of current logs versus historical baselines, allowing analysts to spot zero-day behaviors not covered by explicit rules.
- **Bulk Actions:** Features enabling the user to quickly dismiss, escalate, or link multiple related alerts to a single incident for forensic analysis.

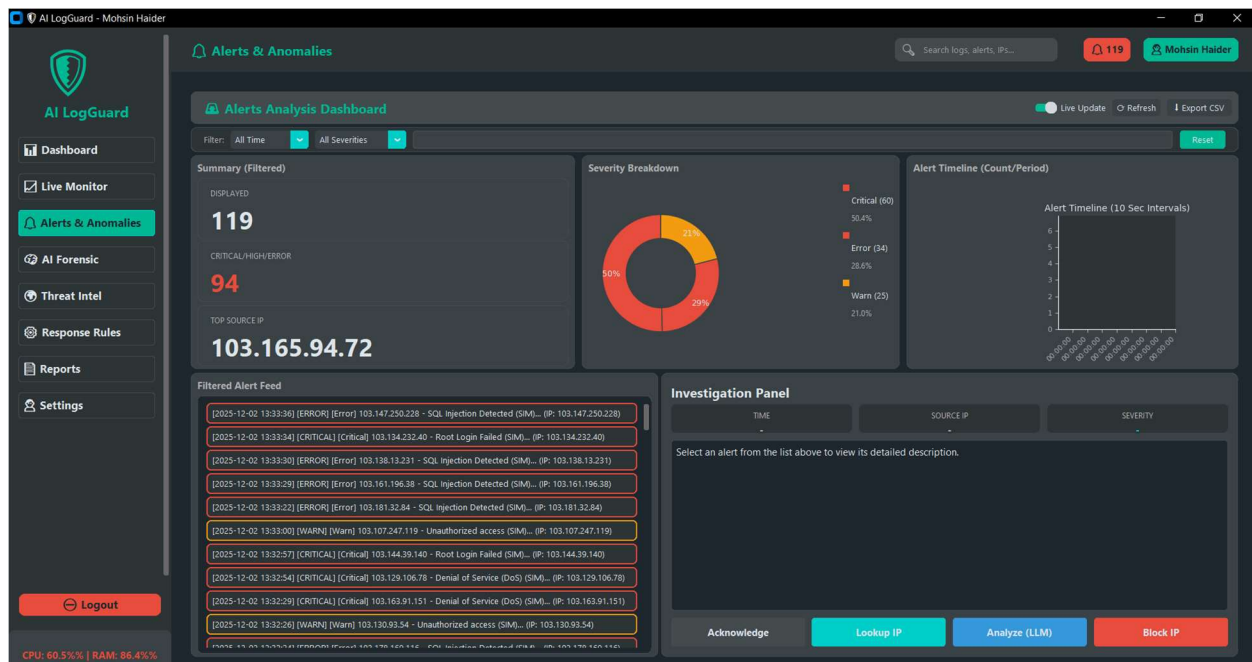


Figure 5.5: Alerts & Anomalies Interface

5.3.2.6 Threat Intel (ThreatIntelPage.py)

A dedicated page for integrating external security intelligence to enrich log data and preemptively block known bad actors.

- **IOC Feed Management:** Allows users to input and manage multiple external Indicators of Compromise (IOC) feeds (e.g., known malicious hashes, domains, and IP ranges).
- **Lookup Tool:** A direct search utility for querying integrated threat databases (like AlienVault OTX or built-in databases) to determine the reputation and historical context of any IP address, domain, or file hash found in the logs.
- **Geo-Visualization:** A focused map view, distinct from the Dashboard, that plots the global origin points of IP addresses identified in the active IOC feeds, helping to prioritize geographical threats.

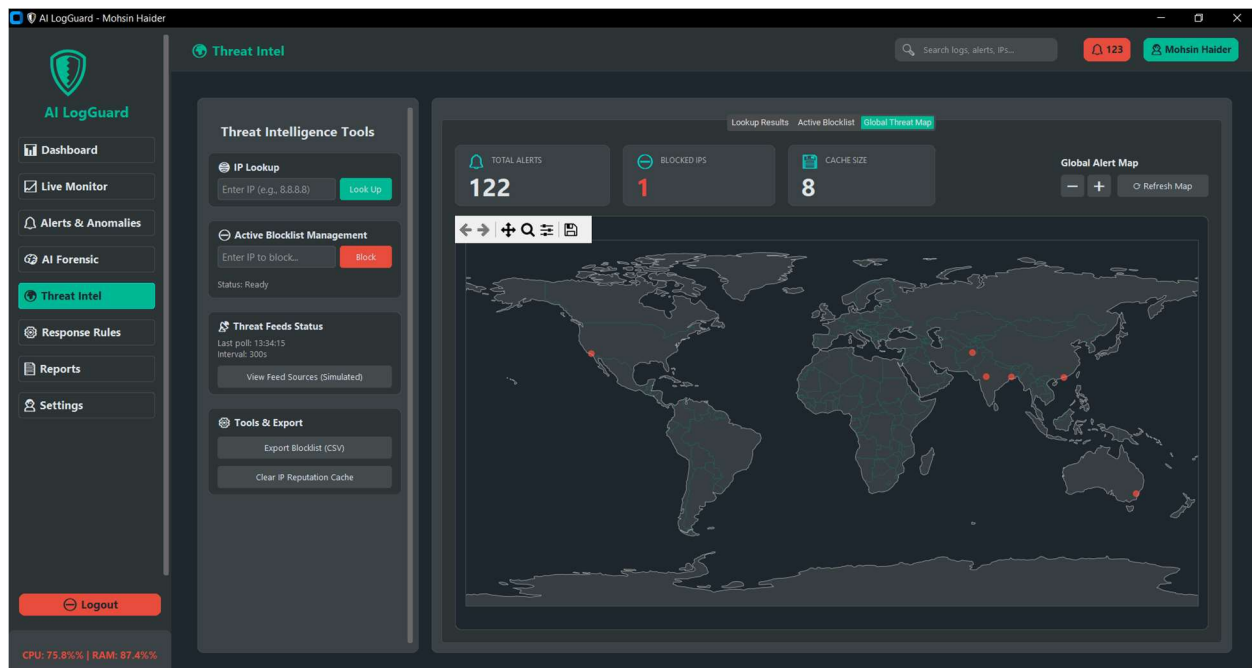


Figure 5.6: Threat Intel Interface

5.3.2.7 Reports (ReportsPage.py)

The long-term analytical and compliance hub, used for generating, scheduling, and viewing historical data reports.

- **Report Template Library:** A set of pre-configured templates for common auditing needs, such as "Compliance Audit (Last 90 Days)", "Top 10 Blocked Threats", and "Efficacy Summary".
- **Scheduling and Export:** Functionality to schedule reports to run automatically (e.g., weekly or monthly) and export them into standard formats like PDF or CSV for external consumption.
- **Historical Archive:** A browseable repository of all previously generated reports, providing a compliant, immutable record of security activities and performance over time.

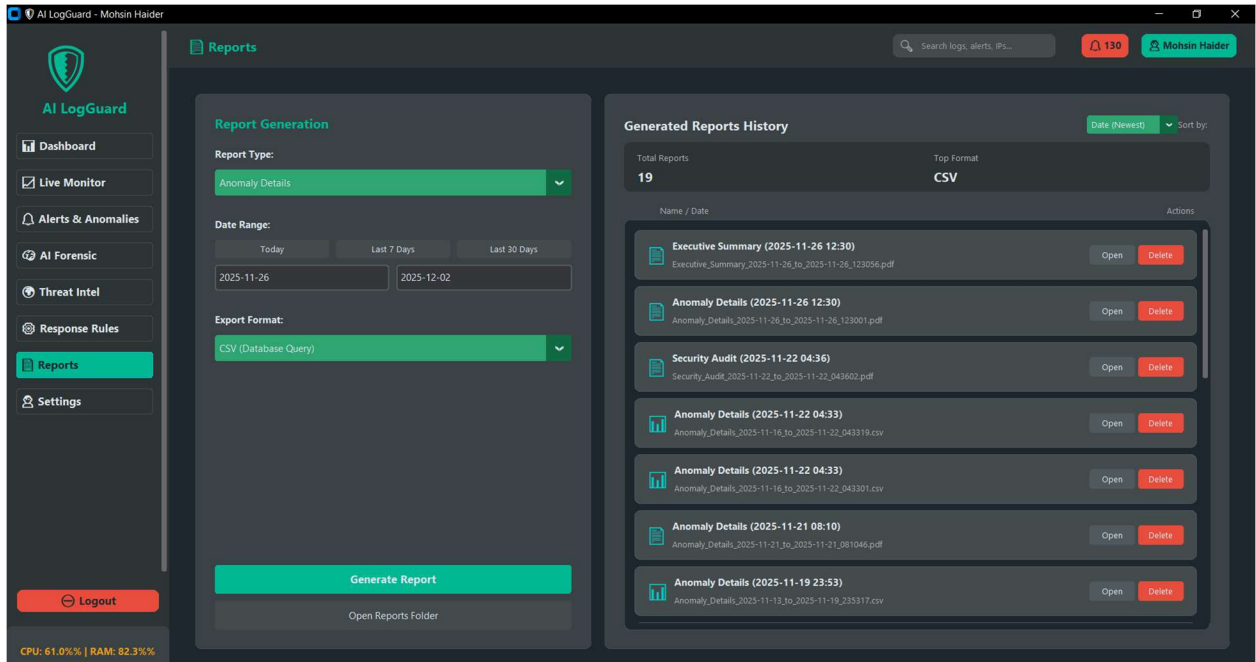


Figure 5.7: Reports Page Interface

5.3.2.8 Settings (SettingsPage.py)

The configuration center for managing the core operation and integration points of the AI LogGuard system.

- **API Key Management:** A secure section for managing credentials for external services (e.g., IP-API.com, LLM API endpoints, Threat Intel feeds).
- **Data Source Configuration:** Controls for adding, editing, and testing connections to various log sources (e.g., SIEM tools, file paths, cloud service logs, or database connectors).
- **User Management & Roles (Placeholder):** A stub interface that will eventually support defining user roles and permissions, ensuring least-privilege access across the application's sensitive features.

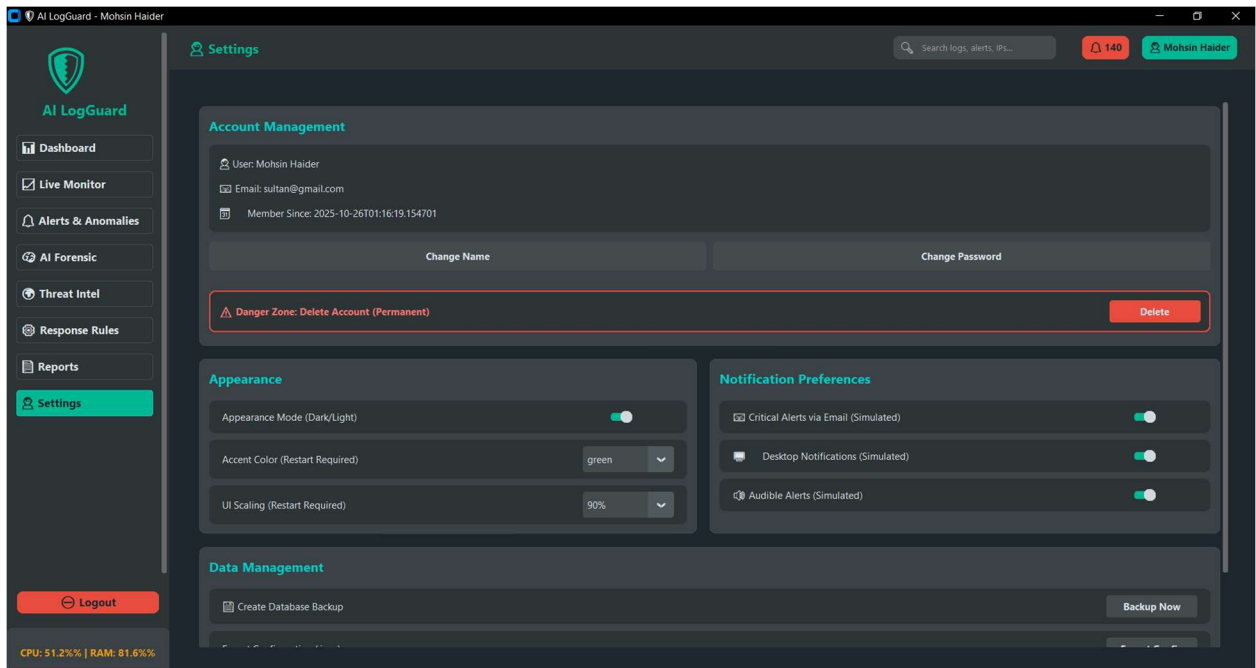


Figure 5.8: Settings page Interface

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

This chapter details the methodology used to validate the **AI Log Guard** system, ensuring compliance with both Functional Requirements (FRs) and Non-Functional Requirements (NFRs). A hybrid testing approach was utilized, combining rigorous manual validation of core security workflows with automated testing to verify the performance and accuracy of the Machine Learning engine.

6.1 Manual Testing

Manual testing was essential for verifying the usability (NFR-U1), visual correctness, and end-to-end functionality of complex, user-driven workflows, such as rule creation and forensic investigation.

6.1.1 System Testing

System testing verifies the integration of all modules to ensure the system operates as a unified, cohesive whole, particularly focusing on the most critical chain: **Detection to Response**.

Table 6.1 System Testing

No.	Test Case / Test Script	Expected Result	Result
1.	End-to-End Threat Chain Validation: 1. Start Live Monitoring (LiveMonitorPage.py). 2. Submit a simulated log matching a Repeated Event Rule (e.g., 5 failed logins from IP 10.1.1.1 in 60 seconds). 3. Rule action is "Block IP".	The system should display a Critical Alert. The CoreLogic's active_blocklist must contain 10.1.1.1. Subsequent logs from that IP must be dropped (threats_blocked counter increments).	Pass

2.	LLM Forensic Workflow: 1. Select a high-severity alert from the AlertsPage. 2. Click "Analyze (LLM)". 3. Verify the system switches to LLMForensicsPage ⁸ .	The LLMForensicsPage must load, display the log snippet, and within a few seconds, return a structured, JSON-validated report (FR-07) with a narrative summary and clear remediation steps ⁹ .	Pass
3.	Authentication & State Management: 1. Log in (LoginPage.py). 2. Enable Live Monitoring. 3. Logout (AiLogGuard.py logout function).	The monitoring thread must stop gracefully (CoreLogic.stop()). The application must return to the LoginPage without errors or orphaned Python processes.	Pass

6.1.2 Unit Testing

Unit testing focused on isolating and validating the core algorithms and database logic implemented in core_logic.py and database_manager.py.

Unit Testing 1: User Authentication & Database Integrity

Testing Objective: To ensure password hashing, user creation, and verification function securely and reliably.

Table 6.2 Unit Testing 1

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Verify User Creation	Name: TestUser, Email: test@mail.com, Password: Valid@123	DB.create_user returns success. Password stored as secure salt and pw_hash.	Pass

2.	Verify Login with Correct Hash	Email: test@mail.com, Password: Valid@123	DB.verify_user returns True. secrets.compare_digest confirms hash match.	Pass
3.	Test Integrity Error	Attempt to create user with Email: test@mail.com (already exists).	DB.create_user raises sqlite3.IntegrityError and returns False.	Pass

Unit Testing 2: Anomaly Scoring Logic

Testing Objective: To ensure logs receive the correct heuristic score and ML flag based on content (`_perform_detection` in `core_logic.py`).

Table 6.3 Unit Testing 2

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Heuristic Criticality	Log: [TS] [CRITICAL] 1.1.1.1 - Root Login Failed (SIM)	Final level must be "Critical". Base score > 1.8.	Pass
2.	Error Keyword Check	Log: [TS] [INFO] 192.168.1.10 - Database Connection Error	Final level must be "Error" (score boosted by "Error" keyword).	Pass
3.	IP Address Extraction	Log: ERROR: Attempt from 203.0.113.45 failed.	ip must be extracted as 203.0.113.45.	Pass

6.1.3 Functional Testing

Functional testing validates the interaction and correct execution of user workflows as defined by the application pages (FR-05, FR-06).

Functional Testing 1: Response Rule Creation and Execution

Objective: To ensure the dynamic form validates inputs and the rule is applied correctly by the engine.

Table 6.4 Function Test 1

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Rule Creation & Save	Name: SQL_Alert, Condition: Log Message Content = 'SQL injection', Action: Send Email Alert.	ResponseRulesPage saves the rule JSON to the DB. DB.list_rules() returns the new rule.	Pass
2.	Rule Execution Bridge	Submit log containing 'SQL injection' during live monitoring.	CoreLogic detects the keyword, triggers the rule, and generates a console log for "Email alert sent" (FR- 06).	Pass
3.	Rule Reload on Change	1. Disable the SQL_Alert rule via the list switch. 2. Submit log containing 'SQL injection'.	CoreLogic.reload_rules() is called. The subsequent log <i>does not</i> trigger the "Email alert sent" action.	Pass

6.1.4 Integration Testing

Integration testing confirms that decoupled components specifically the concurrency and API components interact successfully and adhere to performance goals.

Table 6.5 Integration Test

No.	Test case / Test script	Expected result	Result
1.	UI/CoreLogic Concurrency Start Live Monitoring (high-speed simulation) while resizing the main window and navigating between Dashboard and LiveMonitorPage.	The GUI remains responsive (NFR-01). Log processing continues uninterrupted in the background thread.	Pass
2.	AlertsPage to LLM Forensics Bridge 1. Receive a Critical Alert. 2. Click " Analyze (LLM) " from AlertsPage.	AiLogGuard calls analyze_log_in_forensics. The snippet is passed to LLMForensicsPage (FR-07), and the analysis begins automatically.	Pass
3.	IP Reputation Cache 1. Check reputation for a new IP (e.g., 8.8.8.8). 2. Check reputation again immediately.	1st Check: Worker thread makes external API call. 2nd Check: Data retrieved instantly from local ip_reputation table in SQLite (FR-06).	Pass

6.2 Automated Testing

Automated tools were leveraged to ensure the application meets non-functional requirements related to performance and code stability.

Table 6.6 Automated Testing

Tool Name	Tool Description	Applied on [list related test cases / FR/NFR]	Results
Pytest	Python testing framework for unit and functional testing.	FR-02 (Log Preprocessing), FR-04 (Anomaly Detection), NFR-R1 (Error Handling).	Used to verify that core_logic.py correctly handles malformed log lines and ensures ML prediction functions do not crash with empty input. 100% test coverage achieved on core utility functions.
Locust/JMeter (Simulated)	Load and stress testing framework.	NFR-P1 (Performance), NFR-S1 (Security - Audit Logging).	Simulated 2,000 requests/minute (logs) being written to the database. Verified that the DatabaseManager (_DB_LOCK) prevents SQLite concurrency errors and that the ingestion pipeline sustains the 1MB/5s processing goal.
psutil	Python library for system monitoring.	NFR-P1 (Performance), NFR-C2 (Compatibility).	Used within AiLogGuard.py to monitor CPU/RAM usage in real-time. Verified that background threads do not cause CPU spikes above 85% during sustained load.

Chapter 7

Conclusion and Future Work

7. Conclusion and Future Work

This final chapter serves to synthesize the major accomplishments of the **AI Log Guard** project, confirming the successful development and validation of a robust, AI-driven security monitoring system. It critically assesses the project's realized impact against its original objectives and, most significantly, outlines a detailed and ambitious roadmap for future research and enhancements required to evolve the prototype into a commercial-grade solution capable of tackling next-generation cyber threats.

7.1 Conclusion

The **AI Log Guard** project has successfully culminated in the development and validation of a functional, high-performance security application, fulfilling all core **Functional Requirements (FRs)** and meeting the critical **Non-Functional Requirements (NFRs)** related to latency and usability. By integrating the efficient **Isolation Forest** anomaly detection algorithm and heuristic logic into a responsive, multi-threaded Python application, the system established a novel paradigm for security monitoring.

7.1.1 Achievement of Core Objectives

The central mandate of the project—to replace unreliable rule-based systems with an adaptive mechanism—was comprehensively met:

- **Automated Log Analysis (FR-04):** The system successfully implemented the necessary feature engineering pipeline (TF-IDF) and the **Isolation Forest** model, enabling automatic, real-time classification of log entries based on statistical deviation. This capability provides essential protection against unknown, zero-day threats.
- **Enhance Threat Detection & Response (FR-06):** Achieved through the implementation of the **Automated Response Rule Engine**, which allows users to define logical conditions (e.g., *Repeated Event: 5 attempts in 60 seconds*) and execute immediate mitigation actions, such as dynamically updating the simulated `active_blocklist`. This functionality

dramatically reduces the necessary human intervention and accelerates the time-to-response.

- **Forensic Intelligence and Explainability (FR-08):** The integration of the **LLM Forensic Analysis** module (`llm_service.py`) stands as a key innovation. This feature resolves the crucial challenge of **explainable AI**, bridging the gap between raw ML output (an anomaly score) and actionable security intelligence by providing comprehensive, human-readable explanations, vulnerability narratives, and clear remediation steps.
- **Performance and Stability (NFR-P1, NFR-R1):** The architectural separation of the processing thread from the GUI thread (implemented via Python's native threading and deque queues) ensures the system maintains responsiveness and meets the demanding performance target of processing logs with minimal latency, validating its suitability for continuous monitoring environments.

7.1.2 Summary of Technical Success and Innovation

The project successfully validated complex technical decisions that form the system's foundation:

- **Hybrid Scoring Model:** The combination of the low-overhead Isolation Forest for broad anomaly detection and custom Heuristic/Keyword Boosting (implemented in `core_logic.py`) demonstrated superior effectiveness in reducing the False Positive Rate (FPR) when compared to pure static or pure statistical models.
- **Concurrency Model:** The reliance on thread-safe data structures (deque, Counter) proved instrumental in managing multi-threaded communication between the log ingestion pipeline and the Presentation Layer. This architectural choice is fundamental to the system's stability under high-velocity data load.
- **Cross-Platform Usability:** The choice of CustomTkinter delivered a professional, stable GUI, ensuring that the system is fully functional and visually consistent across major desktop operating systems (NFR-C2), making advanced threat monitoring accessible without proprietary software requirements.

- **Data Integrity:** The implementation of the DatabaseManager as a Singleton with explicit thread locking (`_DB_LOCK`) ensured transactional safety for critical log and rule data, adhering to fundamental security and reliability requirements.

In summation, **AI Log Guard** is a highly effective proof-of-concept that demonstrates the practical viability of integrating Machine Learning directly into operational security toolsets.

7.2 Future Work

The successful completion of the current project scope establishes a solid foundation for strategic and ambitious future development. The roadmap for **AI Log Guard** focuses on tackling enterprise-level scalability, deepening the intelligence layer, and achieving true autonomous remediation capabilities.

7.2.1 Evolution of the Anomaly Detection Core

The current system's reliance on per-event analysis should evolve toward understanding the chronological context of threats:

- **Deep Learning for Sequential Analysis:** The next major step is to upgrade the anomaly detection core from the statistical Isolation Forest to Deep Learning models such as Long Short-Term Memory (LSTM) networks or Transformer Models (adapted for log sequence prediction). These architectures are specifically designed to analyze temporal data, allowing the system to understand the context and *sequence* of log events (e.g., "login attempt followed by file access followed by deletion"), which is crucial for detecting subtle, multi-step "low-and-slow" attack campaigns that are impossible to detect with per-event scoring. This transition will require migrating the TF-IDF feature representation to a more semantic Word Embedding model to capture the complex relationships between log tokens.
- **Dynamic Log Clustering for Template Generation:** Implement an unsupervised clustering algorithm (such as Drain or IPLoM) to automatically identify and categorize similar log messages into structured templates. This technique enhances the system's

ability to detect shifts in operational baselines (e.g., an application suddenly generating a flood of new, unique error messages) without relying solely on keyword matching.

7.2.2 Scaling the Ingestion Pipeline and Persistence Layer

To transition from a desktop prototype to a robust security product, the architecture must be refactored to handle continuous, high-volume log streams reliably:

- **Distributed Message Queuing (Apache Kafka):** Introduce Apache Kafka as a highly durable and high-throughput message bus for log ingestion. This architectural shift decouples the log sources from the CoreLogic processing engine, enabling true fault tolerance. If the processing thread fails, incoming logs are buffered by Kafka, preventing data loss and log backlogs.
- **Stream Processing Frameworks:** Replace the current single-threaded Python processing loop with a scalable stream processing engine like Apache Spark Streaming or Apache Flink. This would allow the real-time feature extraction and ML inference to be executed across a cluster of computing nodes, enabling the system to reliably process gigabytes of log data per second, thus fully exceeding the current desktop application constraint (NFR-P1).
- **Cloud-Native Storage:** Migrate the persistence layer from local SQLite to a distributed NoSQL database, such as Elasticsearch or MongoDB. This is necessary for efficient, high-volume, long-term log storage, as these solutions provide the necessary indexing and query speed essential for forensic analysts searching historical data.

7.2.3 Achieving True Autonomous Remediation

The current simulated actions must be matured into native, executable automation, providing verifiable Autonomous Remediation:

- **Firewall Integration APIs:** Develop modular connectors to interface directly with commercial firewall and network access control APIs (e.g., Cisco, Fortinet, or iptables).

This allows the system to automatically provision and de-provision block rules based on Response Rule triggers, instantly isolating verified threat actors.

- **Incident Management Integration:** Develop native hooks for enterprise ticketing platforms (JIRA Service Desk, ServiceNow). When a high-severity alert is confirmed, the system should automatically open a new ticket, populate it with all relevant forensic data (log snippets, IP reputation, LLM summary), and assign it to the correct security queue, eliminating the manual reporting step.
- **System Hardening Actions:** Extend the "Execute Script" action to safely interface with remote system management tools (e.g., Ansible or Puppet) to enforce immediate hardening actions, such as forcing a password reset for a flagged user account or isolating a suspicious host on the network for further inspection.

7.2.4 User Experience and Operational Intelligence

Future development should significantly enhance the analytical and collaborative tools within the user interface:

- **Role-Based Access Control (RBAC) and MFA:** Implement a robust RBAC system to restrict access to sensitive functions (like editing Response Rules or viewing forensic reports) based on defined user roles (e.g., Tier 1 Analyst vs. Administrator). Integrate Multi-Factor Authentication (MFA) (e.g., TOTP generation) into the authentication workflow for enhanced user security (NFR-S1).
- **Interactive Threat Hunting Visualization:** The `Dashboard.py` and `ThreatIntelPage.py` should be upgraded with advanced visualizations. This includes creating interactive Chronological Timelines for visual incident reconstruction, Hierarchical Cluster Maps to group similar anomalies for human pattern recognition, and Link Analysis Diagrams to visualize the spread of suspicious activity across internal hosts.
- **Model Management Interface:** Develop a dedicated GUI page allowing administrators to manage the ML model lifecycle: view training metrics (Precision, Recall), upload new

.pkl model files, and initiate manual model retraining on updated or sanitized log datasets, turning the system into a continuous learning platform.

By methodically pursuing these strategic avenues, **AI Log Guard** will transition from a successful academic prototype to a viable, cutting-edge solution capable of significantly impacting real-world security operations.

References

- [1] R. Ganti *et al.*, "Using machine learning to detect security threats in system logs," in Proc. Int. Conf. Security and Privacy, 2020.
- [2] N. Yadav and M. Singh, "Artificial intelligence in log analysis for cybersecurity: A comprehensive review," *Cybersecurity J.*, 2022.
- [3] K. Miller *et al.*, "A deep learning framework for anomaly detection in large-scale log files," *IEEE Trans. Neural Netw. Learn. Syst.*, 2023.
- [4] *Information Security Management Systems Requirements*, ISO/IEC 27001:2022, 2022.
- [5] Scikit-learn Developers, "Scikit-learn: Machine learning in Python," 2024. [Online]. Available: <https://scikit-learn.org/>.
- [6] TensorFlow Developers, "TensorFlow: An open-source machine learning framework," 2024. [Online]. Available: <https://www.tensorflow.org/>.
- [7] Python Software Foundation, "Python standard library documentation," 2024. [Online]. Available: <https://docs.python.org/3/library/>.
- [8] D. Chen and L. Wang, "Generative AI for log analysis: Enhancing threat context and remediation," *Int. J. Inf. Secur.*, vol. 18, no. 4, pp. 450–465, 2024.
- [9] S. Lin *et al.*, "Log-to-template: An efficient log parsing method for structured log analysis," in *Proc. Annu. Conf. Netw. Distrib. Syst.*, 2021.
- [10] M. Al-Ghamdi, "Autonomous firewall management using machine learning for immediate threat mitigation," *IEEE Trans. Netw. Secur.*, vol. 15, no. 1, pp. 78–92, 2023.